

As you can see, Taco is a straightforward Java domain object with a couple of properties. Like Ingredient, the Taco class is annotated with `@Data` to have Lombok automatically generate essential JavaBean methods for you at compile time.

Now that we have defined Ingredient and Taco, we need one more domain class that defines how customers specify the tacos that they want to order, along with payment and delivery information. That's the job of the `TacoOrder` class, shown here.

Listing 2.3 A domain object for taco orders

```
package tacos;
import java.util.List;
import java.util.ArrayList;
import lombok.Data;

@Data
public class TacoOrder {

    private String deliveryName;
    private String deliveryStreet;
    private String deliveryCity;
    private String deliveryState;
    private String deliveryZip;
    private String ccNumber;
    private String ccExpiration;
    private String ccCVV;

    private List<Taco> tacos = new ArrayList<>();

    public void addTaco(Taco taco) {
        this.tacos.add(taco);
    }
}
```

Aside from having more properties than either Ingredient or Taco, there's nothing particularly new to discuss about `TacoOrder`. It's a simple domain class with nine properties: five for delivery information, three for payment information, and one that is the list of Taco objects that make up the order. There's also an `addTaco()` method that's added for the convenience of adding tacos to the order.

Now that the domain types are defined, we're ready to put them to work. Let's add a few controllers to handle web requests in the application.

2.1.2 *Creating a controller class*

Controllers are the major players in Spring's MVC framework. Their primary job is to handle HTTP requests and either hand off a request to a view to render HTML (browser-displayed) or write data directly to the body of a response (RESTful). In this chapter, we're focusing on the kinds of controllers that use views to produce content for web browsers. When we get to chapter 7, we'll look at writing controllers that handle requests in a REST API.

For the Taco Cloud application, you need a simple controller that will do the following:

- Handle HTTP GET requests where the request path is /design
- Build a list of ingredients
- Hand off the request and the ingredient data to a view template to be rendered as HTML and sent to the requesting web browser

The DesignTacoController class in the next listing addresses those requirements.

Listing 2.4 The beginnings of a Spring controller class

```
package tacos.web;

import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.SessionAttributes;

import lombok.extern.slf4j.Slf4j;
import tacos.Ingredient;
import tacos.Ingredient.Type;
import tacos.Taco;

@Slf4j
@Controller
@RequestMapping("/design")
@SessionAttributes("tacoOrder")
public class DesignTacoController {

    @ModelAttribute
    public void addIngredientsToModel(Model model) {
        List<Ingredient> ingredients = Arrays.asList(
            new Ingredient("FLTO", "Flour Tortilla", Type.WRAP),
            new Ingredient("COTO", "Corn Tortilla", Type.WRAP),
            new Ingredient("GRBF", "Ground Beef", Type.PROTEIN),
            new Ingredient("CARN", "Carnitas", Type.PROTEIN),
            new Ingredient("TMTO", "Diced Tomatoes", Type.VEGGIES),
            new Ingredient("LETC", "Lettuce", Type.VEGGIES),
            new Ingredient("CHED", "Cheddar", Type.CHEESE),
            new Ingredient("JACK", "Monterrey Jack", Type.CHEESE),
            new Ingredient("SLSA", "Salsa", Type.SAUCE),
            new Ingredient("SRCR", "Sour Cream", Type.SAUCE)
        );

        Type[] types = Ingredient.Type.values();
        for (Type type : types) {
            model.addAttribute(type.toString().toLowerCase(),
```

```

        filterByType(ingredients, type));
    }
}

@ModelAttribute(name = "tacoOrder")
public TacoOrder order() {
    return new TacoOrder();
}

@ModelAttribute(name = "taco")
public Taco taco() {
    return new Taco();
}

@GetMapping
public String showDesignForm() {
    return "design";
}

private Iterable<Ingredient> filterByType(
    List<Ingredient> ingredients, Type type) {
    return ingredients
        .stream()
        .filter(x -> x.getType().equals(type))
        .collect(Collectors.toList());
}
}

```

The first thing to note about `DesignTacoController` is the set of annotations applied at the class level. The first, `@Slf4j`, is a Lombok-provided annotation that, at compilation time, will automatically generate an SLF4J (Simple Logging Facade for Java, <https://www.slf4j.org/>) Logger static property in the class. This modest annotation has the same effect as if you were to explicitly add the following lines within the class:

```

private static final org.slf4j.Logger log =
    org.slf4j.LoggerFactory.getLogger(DesignTacoController.class);

```

You'll make use of this Logger a little later.

The next annotation applied to `DesignTacoController` is `@Controller`. This annotation serves to identify this class as a controller and to mark it as a candidate for component scanning, so that Spring will discover it and automatically create an instance of `DesignTacoController` as a bean in the Spring application context.

`DesignTacoController` is also annotated with `@RequestMapping`. The `@RequestMapping` annotation, when applied at the class level, specifies the kind of requests that this controller handles. In this case, it specifies that `DesignTacoController` will handle requests whose path begins with `/design`.

Finally, you see that `DesignTacoController` is annotated with `@SessionAttributes("tacoOrder")`. This indicates that the `TacoOrder` object that is put into the model a little later in the class should be maintained in session. This is important because the

creation of a taco is also the first step in creating an order, and the order we create will need to be carried in the session so that it can span multiple requests.

HANDLING A GET REQUEST

The class-level `@RequestMapping` specification is refined with the `@GetMapping` annotation that adorns the `showDesignForm()` method. `@GetMapping`, paired with the class-level `@RequestMapping`, specifies that when an HTTP GET request is received for `/design`, Spring MVC will call `showDesignForm()` to handle the request.

`@GetMapping` is just one member of a family of request-mapping annotations. Table 2.1 lists all of the request-mapping annotations available in Spring MVC.

Table 2.1 Spring MVC request-mapping annotations

Annotation	Description
<code>@RequestMapping</code>	General-purpose request handling
<code>@GetMapping</code>	Handles HTTP GET requests
<code>@PostMapping</code>	Handles HTTP POST requests
<code>@PutMapping</code>	Handles HTTP PUT requests
<code>@DeleteMapping</code>	Handles HTTP DELETE requests
<code>@PatchMapping</code>	Handles HTTP PATCH requests

When `showDesignForm()` handles a GET request for `/design`, it doesn't really do much. The main thing it does is return a `String` value of `"design"`, which is the logical name of the view that will be used to render the model to the browser. But before it does that, it also populates the given `Model` with an empty `Taco` object under a key whose name is `"design"`. This will enable the form to have a blank slate on which to create a taco masterpiece.

It would seem that a GET request to `/design` doesn't do much. But on the contrary, there's a bit more involved than what is found in the `showDesignForm()` method. You'll also notice a method named `addIngredientsToModel()` that is annotated with `@ModelAttribute`. This method will also be invoked when a request is handled and will construct a list of `Ingredient` objects to be put into the model. The list is hard-coded for now. When we get to chapter 3, you'll pull the list of available taco ingredients from a database.

Once the list of ingredients is ready, the next few lines of `addIngredientsToModel()` filters the list by ingredient type using a helper method named `filterByType()`. A list of ingredient types is then added as an attribute to the `Model` object that will be passed into `showDesignForm()`. `Model` is an object that ferries data between a controller and whatever view is charged with rendering that data. Ultimately, data that's placed in `Model` attributes is copied into the servlet request attributes, where the view can find them and use them to render a page in the user's browser.

Following `addIngredientsToModel()` are two more methods that are also annotated with `@ModelAttribute`. These methods are much simpler and create only a new `TacoOrder` and `Taco` object to place into the model. The `TacoOrder` object, referred to earlier in the `@SessionAttributes` annotation, holds state for the order being built as the user creates tacos across multiple requests. The `Taco` object is placed into the model so that the view rendered in response to the GET request for `/design` will have a non-null object to display.

Your `DesignTacoController` is really starting to take shape. If you were to run the application now and point your browser at the `/design` path, the `DesignTacoController`'s `showDesignForm()` and `addIngredientsToModel()` would be engaged, placing ingredients and an empty `Taco` into the model before passing the request on to the view. But because you haven't defined the view yet, the request would take a horrible turn, resulting in an HTTP 500 (Internal Server Error) error. To fix that, let's switch our attention to the view where the data will be decorated with HTML to be presented in the user's web browser.

2.1.3 *Designing the view*

After the controller is finished with its work, it's time for the view to get going. Spring offers several great options for defining views, including JavaServer Pages (JSP), Thymeleaf, FreeMarker, Mustache, and Groovy-based templates. For now, we'll use Thymeleaf, the choice we made in chapter 1 when starting the project. We'll consider a few of the other options in section 2.5.

We have already added Thymeleaf as a dependency in chapter 1. At run time, Spring Boot autoconfiguration sees that Thymeleaf is in the classpath and automatically creates the beans that support Thymeleaf views for Spring MVC.

View libraries such as Thymeleaf are designed to be decoupled from any particular web framework. As such, they're unaware of Spring's model abstraction and are unable to work with the data that the controller places in `Model`. But they can work with servlet request attributes. Therefore, before Spring hands the request over to a view, it copies the model data into request attributes that Thymeleaf and other view-templating options have ready access to.

Thymeleaf templates are just HTML with some additional element attributes that guide a template in rendering request data. For example, if there were a request attribute whose key is `"message"`, and you wanted it to be rendered into an HTML `<p>` tag by Thymeleaf, you'd write the following in your Thymeleaf template:

```
<p th:text="${message}">placeholder message</p>
```

When the template is rendered into HTML, the body of the `<p>` element will be replaced with the value of the servlet request attribute whose key is `"message"`. The `th:text` attribute is a Thymeleaf namespace attribute that performs the replacement. The `${}` operator tells it to use the value of a request attribute (`"message"`, in this case).

Thymeleaf also offers another attribute, `th:each`, that iterates over a collection of elements, rendering the HTML once for each item in the collection. This attribute will come in handy as you design your view to list taco ingredients from the model. For example, to render just the list of "wrap" ingredients, you can use the following snippet of HTML:

```
<h3>Designate your wrap:</h3>
<div th:each="ingredient : ${wrap}">
  <input th:field="*{ingredients}" type="checkbox"
        th:value="${ingredient.id}"/>
  <span th:text="${ingredient.name}">INGREDIENT</span><br/>
</div>
```

Here, you use the `th:each` attribute on the `<div>` tag to repeat rendering of the `<div>` once for each item in the collection found in the `wrap` request attribute. On each iteration, the ingredient item is bound to a Thymeleaf variable named `ingredient`.

Inside the `<div>` element are a check box `<input>` element and a `<span>` element to provide a label for the check box. The check box uses Thymeleaf's `th:value` to set the rendered `<input>` element's value attribute to the value found in the ingredient's `id` property. The `th:field` attribute ultimately sets the `<input>` element's name attribute and is used to remember whether or not the check box is checked. When we add validation later, this will ensure that the check box maintains its state should the form need to be redisplayed after a validation error. The `<span>` element uses `th:text` to replace the "INGREDIENT" placeholder text with the value of the ingredient's name property.

When rendered with actual model data, one iteration of that `<div>` loop might look like this:

```
<div>
  <input name="ingredients" type="checkbox" value="FLTO" />
  <span>Flour Tortilla</span><br/>
</div>
```

Ultimately, the preceding Thymeleaf snippet is just part of a larger HTML form through which your taco artist users will submit their tasty creations. The complete Thymeleaf template, including all ingredient types and the form, is shown in the following listing.

Listing 2.5 The complete design-a-taco page

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:th="http://www.thymeleaf.org">
  <head>
    <title>Taco Cloud</title>
    <link rel="stylesheet" th:href="@{/styles.css}" />
  </head>
```

```

<body>
  <h1>Design your taco!</h1>
  

  <form method="POST" th:object="${taco}">
    <div class="grid">
      <div class="ingredient-group" id="wraps">
        <h3>Designate your wrap:</h3>
        <div th:each="ingredient : ${wrap}">
          <input th:field="**{ingredients}" type="checkbox"
            th:value="${ingredient.id}"/>
          <span th:text="${ingredient.name}">INGREDIENT</span><br/>
        </div>
      </div>

      <div class="ingredient-group" id="proteins">
        <h3>Pick your protein:</h3>
        <div th:each="ingredient : ${protein}">
          <input th:field="**{ingredients}" type="checkbox"
            th:value="${ingredient.id}"/>
          <span th:text="${ingredient.name}">INGREDIENT</span><br/>
        </div>
      </div>

      <div class="ingredient-group" id="cheeses">
        <h3>Choose your cheese:</h3>
        <div th:each="ingredient : ${cheese}">
          <input th:field="**{ingredients}" type="checkbox"
            th:value="${ingredient.id}"/>
          <span th:text="${ingredient.name}">INGREDIENT</span><br/>
        </div>
      </div>

      <div class="ingredient-group" id="veggies">
        <h3>Determine your veggies:</h3>
        <div th:each="ingredient : ${veggies}">
          <input th:field="**{ingredients}" type="checkbox"
            th:value="${ingredient.id}"/>
          <span th:text="${ingredient.name}">INGREDIENT</span><br/>
        </div>
      </div>

      <div class="ingredient-group" id="sauces">
        <h3>Select your sauce:</h3>
        <div th:each="ingredient : ${sauce}">
          <input th:field="**{ingredients}" type="checkbox"
            th:value="${ingredient.id}"/>
          <span th:text="${ingredient.name}">INGREDIENT</span><br/>
        </div>
      </div>
    </div>
  </div>

```

```
<h3>Name your taco creation:</h3>
<input type="text" th:field="*{name}"/>
<br/>

<button>Submit Your Taco</button>
</div>
</form>
</body>
</html>
```

As you can see, you repeat the `<div>` snippet for each of the types of ingredients, and you include a Submit button and field where the user can name their creation.

It's also worth noting that the complete template includes the Taco Cloud logo image and a `<link>` reference to a stylesheet.² In both cases, Thymeleaf's `@{ }` operator is used to produce a context-relative path to the static artifacts that these tags are referencing. As you learned in chapter 1, static content in a Spring Boot application is served from the `/static` directory at the root of the classpath.

Now that your controller and view are complete, you can fire up the application to see the fruits of your labor. We have many ways to run a Spring Boot application. In chapter 1, I showed you how to run the application by clicking the Start button in the Spring Boot Dashboard. No matter how you fire up the Taco Cloud application, once it starts, point your browser to `http://localhost:8080/design`. You should see a page that looks something like figure 2.3.

It's looking good! A taco artist visiting your site is presented with a form containing a palette of taco ingredients from which they can create their masterpiece. But what happens when they click the Submit Your Taco button?

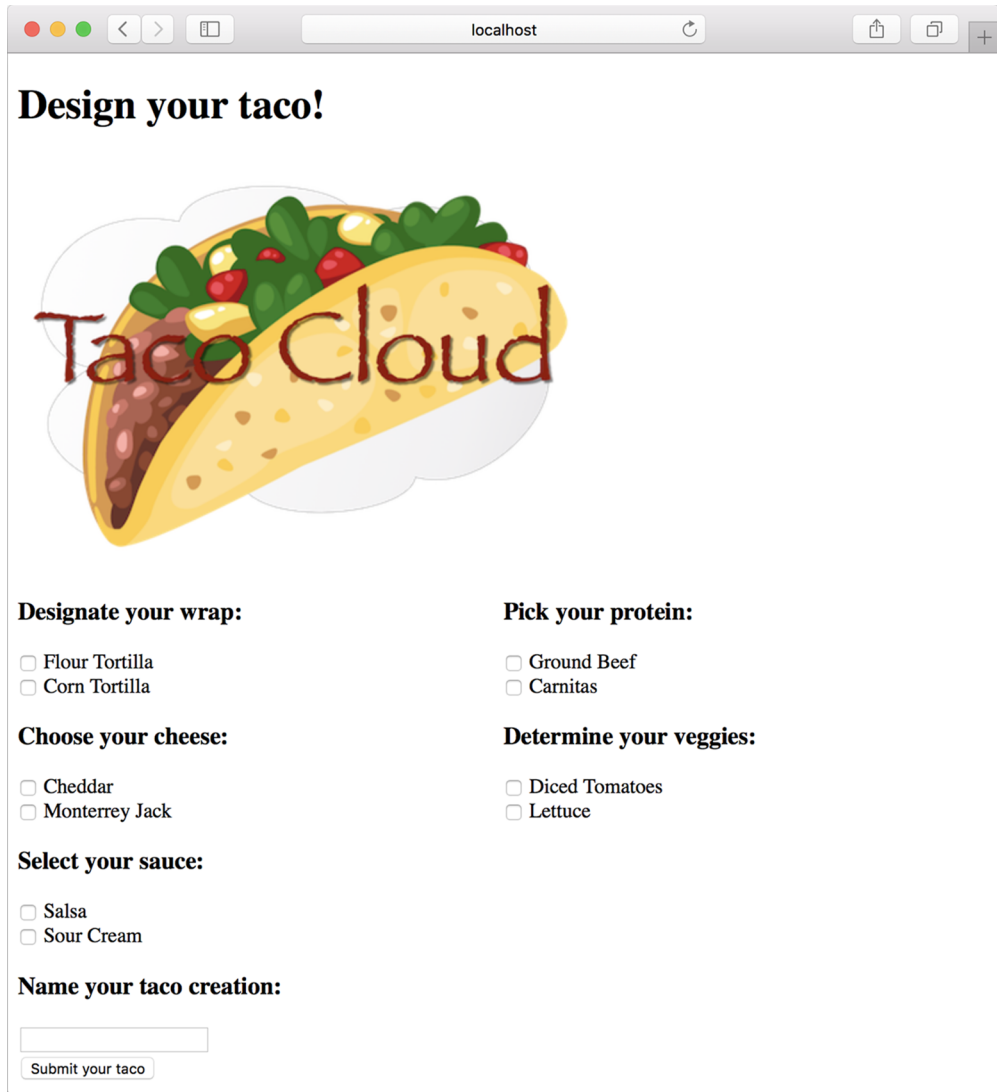
Your `DesignTacoController` isn't yet ready to accept taco creations. If the design form is submitted, the user will be presented with an error. (Specifically, it will be an HTTP 405 error: Request Method "POST" Not Supported.) Let's fix that by writing some more controller code that handles form submission.

2.2 Processing form submission

If you take another look at the `<form>` tag in your view, you can see that its `method` attribute is set to `POST`. Moreover, the `<form>` doesn't declare an `action` attribute. This means that when the form is submitted, the browser will gather all the data in the form and send it to the server in an HTTP POST request to the same path for which a GET request displayed the form—the `/design` path.

Therefore, you need a controller handler method on the receiving end of that POST request. You need to write a new handler method in `DesignTacoController` that handles a POST request for `/design`.

² The contents of the stylesheet aren't relevant to our discussion; it contains only styling to present the ingredients in two columns instead of one long list of ingredients.



Design your taco!



Designate your wrap:

- ☐ Flour Tortilla
- ☐ Corn Tortilla

Choose your cheese:

- ☐ Cheddar
- ☐ Monterrey Jack

Select your sauce:

- ☐ Salsa
- ☐ Sour Cream

Name your taco creation:

Pick your protein:

- ☐ Ground Beef
- ☐ Carnitas

Determine your veggies:

- ☐ Diced Tomatoes
- ☐ Lettuce

Figure 2.3 The rendered taco design page

In listing 2.4, you used the `@GetMapping` annotation to specify that the `showDesignForm()` method should handle HTTP GET requests for `/design`. Just like `@GetMapping` handles GET requests, you can use `@PostMapping` to handle POST requests. For handling taco design submissions, add the `processTaco()` method in the following listing to `DesignTacoController`.

Listing 2.6 Handling POST requests with @PostMapping

```
@PostMapping
public String processTaco(Taco taco,
    @ModelAttribute TacoOrder tacoOrder) {
    tacoOrder.addTaco(taco);
    log.info("Processing taco: {}", taco);

    return "redirect:/orders/current";
}
```

As applied to the `processTaco()` method, `@PostMapping` coordinates with the class-level `@RequestMapping` to indicate that `processTaco()` should handle POST requests for `/design`. This is precisely what you need to process a taco artist's submitted creations.

When the form is submitted, the fields in the form are bound to properties of a `Taco` object (whose class is shown in the next listing) that's passed as a parameter into `processTaco()`. From there, the `processTaco()` method can do whatever it wants with the `Taco` object. In this case, it adds the `Taco` to the `TacoOrder` object passed as a parameter to the method and then logs it. The `@ModelAttribute` applied to the `TacoOrder` parameter indicates that it should use the `TacoOrder` object that was placed into the model via the `@ModelAttribute`-annotated `order()` method shown earlier in listing 2.4.

If you look back at the form in listing 2.5, you'll see several checkbox elements, all with the name `ingredients`, and a text input element named `name`. Those fields in the form correspond directly to the `ingredients` and `name` properties of the `Taco` class.

The `name` field on the form needs to capture only a simple textual value. Thus the `name` property of `Taco` is of type `String`. The `ingredients` check boxes also have textual values, but because zero or many of them may be selected, the `ingredients` property that they're bound to is a `List<Ingredient>` that will capture each of the chosen ingredients.

But wait. If the `ingredients` check boxes have textual (e.g., `String`) values, but the `Taco` object represents a list of ingredients as `List<Ingredient>`, then isn't there a mismatch? How can a textual list like `["FLTO", "GRBF", "LETC"]` be bound to a list of `Ingredient` objects that are richer objects containing not only an ID but also a descriptive name and ingredient type?

That's where a converter comes in handy. A converter is any class that implements Spring's `Converter` interface and implements its `convert()` method to take one value and convert it to another. To convert a `String` to an `Ingredient`, we'll use the `IngredientByIdConverter` as follows.

Listing 2.7 Converting strings to ingredients

```
package tacos.web;

import java.util.HashMap;
import java.util.Map;
```

```

import org.springframework.core.convert.converter.Converter;
import org.springframework.stereotype.Component;

import tacos.Ingredient;
import tacos.Ingredient.Type;

@Component
public class IngredientByIdConverter implements Converter<String, Ingredient> {

    private Map<String, Ingredient> ingredientMap = new HashMap<>();

    public IngredientByIdConverter() {
        ingredientMap.put("FLTO",
            new Ingredient("FLTO", "Flour Tortilla", Type.WRAP));
        ingredientMap.put("COTO",
            new Ingredient("COTO", "Corn Tortilla", Type.WRAP));
        ingredientMap.put("GRBF",
            new Ingredient("GRBF", "Ground Beef", Type.PROTEIN));
        ingredientMap.put("CARN",
            new Ingredient("CARN", "Carnitas", Type.PROTEIN));
        ingredientMap.put("TMTO",
            new Ingredient("TMTO", "Diced Tomatoes", Type.VEGGIES));
        ingredientMap.put("LETC",
            new Ingredient("LETC", "Lettuce", Type.VEGGIES));
        ingredientMap.put("CHED",
            new Ingredient("CHED", "Cheddar", Type.CHEESE));
        ingredientMap.put("JACK",
            new Ingredient("JACK", "Monterrey Jack", Type.CHEESE));
        ingredientMap.put("SLSA",
            new Ingredient("SLSA", "Salsa", Type.SAUCE));
        ingredientMap.put("SRCR",
            new Ingredient("SRCR", "Sour Cream", Type.SAUCE));
    }

    @Override
    public Ingredient convert(String id) {
        return ingredientMap.get(id);
    }
}

```

Because we don't yet have a database from which to pull `Ingredient` objects, the constructor of `IngredientByIdConverter` creates a `Map` keyed on a `String` that is the ingredient ID and whose values are `Ingredient` objects. In chapter 3, we'll adapt this converter to pull the ingredient data from a database instead of being hardcoded like this. The `convert()` method then simply takes a `String` that is the ingredient ID and uses it to look up the `Ingredient` from the map.

Notice that the `IngredientByIdConverter` is annotated with `@Component` to make it discoverable as a bean in the Spring application context. Spring Boot autoconfiguration will discover this, and any other `Converter` beans, and will automatically register them with Spring MVC to be used when the conversion of request parameters to bound properties is needed.

For now, the `processTaco()` method does nothing with the `Taco` object. In fact, it doesn't do much of anything at all. That's OK. In chapter 3, you'll add some persistence logic that will save the submitted `Taco` to a database.

Just as with the `showDesignForm()` method, `processTaco()` finishes by returning a `String` value. And just like `showDesignForm()`, the value returned indicates a view that will be shown to the user. But what's different is that the value returned from `processTaco()` is prefixed with `"redirect:"`, indicating that this is a redirect view. More specifically, it indicates that after `processTaco()` completes, the user's browser should be redirected to the relative path `/orders/current`.

The idea is that after creating a `taco`, the user will be redirected to an order form from which they can place an order to have their taco creations delivered. But you don't yet have a controller that will handle a request for `/orders/current`.

Given what you now know about `@Controller`, `@RequestMapping`, and `@GetMapping`, you can easily create such a controller. It might look something like the following listing.

Listing 2.8 A controller to present a taco order form

```
package tacos.web;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.SessionAttributes;
import org.springframework.web.bind.support.SessionStatus;

import lombok.extern.slf4j.Slf4j;
import tacos.TacoOrder;

@Slf4j
@Controller
@RequestMapping("/orders")
@SessionAttributes("tacoOrder")
public class OrderController {

    @GetMapping("/current")
    public String orderForm() {
        return "orderForm";
    }
}
```

Once again, you use Lombok's `@Slf4j` annotation to create a free `SLF4J` Logger object at compile time. You'll use this Logger in a moment to log the details of the order that's submitted.

The class-level `@RequestMapping` specifies that any request-handling methods in this controller will handle requests whose path begins with `/orders`. When combined with the method-level `@GetMapping`, it specifies that the `orderForm()` method will handle HTTP GET requests for `/orders/current`.

As for the `orderForm()` method itself, it's extremely basic, only returning a logical view name of `orderForm`. Once you have a way to persist taco creations to a database in chapter 3, you'll revisit this method and modify it to populate the model with a list of Taco objects to be placed in the order.

The `orderForm` view is provided by a Thymeleaf template named `orderForm.html`, which is shown next.

Listing 2.9 A taco order form view

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:th="http://www.thymeleaf.org">
  <head>
    <title>Taco Cloud</title>
    <link rel="stylesheet" th:href="@{/styles.css}" />
  </head>

  <body>

    <form method="POST" th:action="@{/orders}" th:object="${tacoOrder}">
      <h1>Order your taco creations!</h1>

      <h3>Your tacos in this order:</h3>
      <a th:href="@{/design}" id="another">Design another taco</a><br/>
      <ul>
        <li th:each="taco : ${tacoOrder.tacos}">
          <span th:text="${taco.name}">taco name</span></li>
      </ul>

      <h3>Deliver my taco masterpieces to...</h3>
      <label for="deliveryName">Name: </label>
      <input type="text" th:field="*{deliveryName}" />
      <br/>

      <label for="deliveryStreet">Street address: </label>
      <input type="text" th:field="*{deliveryStreet}" />
      <br/>

      <label for="deliveryCity">City: </label>
      <input type="text" th:field="*{deliveryCity}" />
      <br/>

      <label for="deliveryState">State: </label>
      <input type="text" th:field="*{deliveryState}" />
      <br/>

      <label for="deliveryZip">Zip code: </label>
      <input type="text" th:field="*{deliveryZip}" />
      <br/>

      <h3>Here's how I'll pay...</h3>
      <label for="ccNumber">Credit Card #: </label>
```

```

<input type="text" th:field="*{ccNumber}"/>
<br/>

<label for="ccExpiration">Expiration: </label>
<input type="text" th:field="*{ccExpiration}"/>
<br/>

<label for="ccCVV">CVV: </label>
<input type="text" th:field="*{ccCVV}"/>
<br/>

<input type="submit" value="Submit Order"/>
</form>
</body>
</html>

```

For the most part, the `orderForm.html` view is typical HTML/Thymeleaf content, with very little of note. It starts by listing the tacos that were added to the order. It uses Thymeleaf's `th:each` to cycle through the order's `tacos` property as it creates the list. Then it renders the order form.

But notice that the `<form>` tag here is different from the `<form>` tag used in listing 2.5 in that it also specifies a form action. Without an action specified, the form would submit an HTTP POST request back to the same URL that presented the form. But here, you specify that the form should be POSTed to `/orders` (using Thymeleaf's `@{...}` operator for a context-relative path).

Therefore, you're going to need to add another method to your `OrderController` class that handles POST requests for `/orders`. You won't have a way to persist orders until the next chapter, so you'll keep it simple here—something like what you see in the next listing.

Listing 2.10 Handling a taco order submission

```

@PostMapping
public String processOrder(TacoOrder order,
    SessionStatus sessionStatus) {
    log.info("Order submitted: {}", order);
    sessionStatus.setComplete();

    return "redirect:/";
}

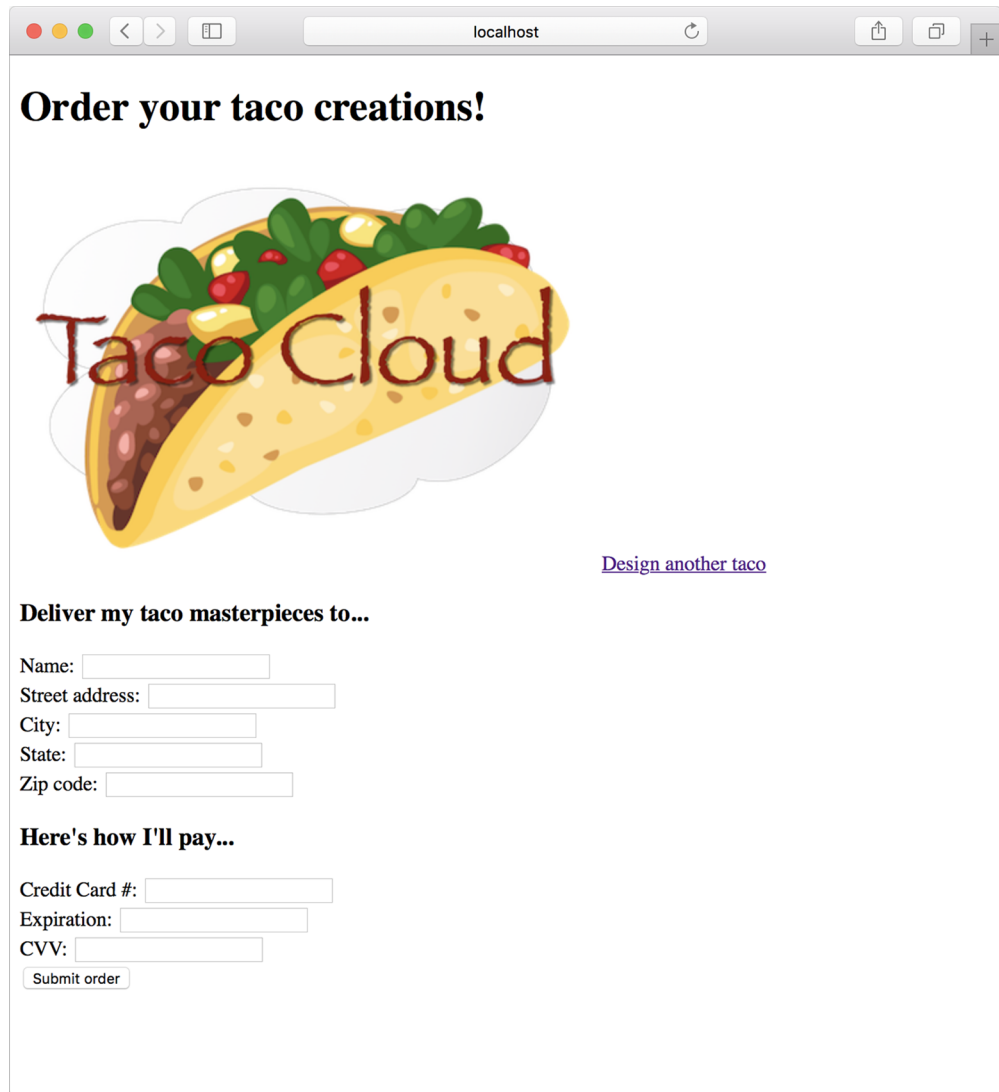
```

When the `processOrder()` method is called to handle a submitted order, it's given a `TacoOrder` object whose properties are bound to the submitted form fields. `TacoOrder`, much like `Taco`, is a fairly straightforward class that carries order information.

In the case of this `processOrder()` method, the `TacoOrder` object is simply logged. We'll see how to persist it to a database in the next chapter. But before `processOrder()` is done, it also calls `setComplete()` on the `SessionStatus` object passed in as a parameter. The `TacoOrder` object was initially created and placed into the session

when the user created their first taco. By calling `setComplete()`, we are ensuring that the session is cleaned up and ready for a new order the next time the user creates a taco.

Now that you've developed an `OrderController` and the order form view, you're ready to try it out. Open your browser to `http://localhost:8080/design`, select some ingredients for your taco, and click the Submit Your Taco button. You should see a form similar to what's shown in figure 2.4.



The screenshot shows a web browser window with the address bar set to 'localhost'. The page content includes a heading 'Order your taco creations!', a large illustration of a taco with the text 'Taco Cloud' overlaid, and a link 'Design another taco'. Below this is a section titled 'Deliver my taco masterpieces to...' with input fields for Name, Street address, City, State, and Zip code. Another section titled 'Here's how I'll pay...' contains input fields for Credit Card #, Expiration, and CVV, followed by a 'Submit order' button.

Order your taco creations!



[Design another taco](#)

Deliver my taco masterpieces to...

Name:

Street address:

City:

State:

Zip code:

Here's how I'll pay...

Credit Card #:

Expiration:

CVV:

Figure 2.4 The taco order form

Fill in some fields in the form, and press the Submit Order button. As you do, keep an eye on the application logs to see your order information. When I tried it, the log entry looked something like this (reformatted to fit the width of this page):

```
Order submitted: TacoOrder(deliveryName=Craig Walls, deliveryStreet=1234 7th
Street, deliveryCity=Somewhere, deliveryState=Who knows?,
deliveryZip=zipzap, ccNumber=Who can guess?, ccExpiration=Some day,
ccCVV=See-vee-vee, tacos=[Taco(name=Awesome Sauce, ingredients=[
Ingredient(id=FLTO, name=Flour Tortilla, type=WRAP), Ingredient(id=GRBF,
name=Ground Beef, type=PROTEIN), Ingredient(id=CHED, name=Cheddar,
type=CHEESE), Ingredient(id=TMTO, name=Diced Tomatoes, type=VEGGIES),
Ingredient(id=SLSA, name=Salsa, type=SAUCE), Ingredient(id=SRCR,
name=Sour Cream, type=SAUCE)]), Taco(name=Quesoriffic, ingredients=
[Ingredient(id=FLTO, name=Flour Tortilla, type=WRAP), Ingredient(id=CHED,
name=Cheddar, type=CHEESE), Ingredient(id=JACK, name=Monterrey Jack,
type=CHEESE), Ingredient(id=TMTO, name=Diced Tomatoes, type=VEGGIES),
Ingredient(id=SRCR, name=Sour Cream, type=SAUCE)])])]
```

It appears that the `processOrder()` method did its job, handling the form submission by logging details about the order. But if you look carefully at the log entry from my test order, you can see that it let a little bit of bad information get in. Most of the fields in the form contained data that couldn't possibly be correct. Let's add some validation to ensure that the data provided at least resembles the kind of information required.

2.3 Validating form input

When designing a new taco creation, what if the user selects no ingredients or fails to specify a name for their creation? When submitting the order, what if the user fails to fill in the required address fields? Or what if they enter a value into the credit card field that isn't even a valid credit card number?

As things stand now, nothing will stop the user from creating a taco without any ingredients or with an empty delivery address, or even submitting the lyrics to their favorite song as the credit card number. That's because you haven't yet specified how those fields should be validated.

One way to perform form validation is to litter the `processTaco()` and `processOrder()` methods with a bunch of `if/then` blocks, checking each and every field to ensure that it meets the appropriate validation rules. But that would be cumbersome and difficult to read and debug.

Fortunately, Spring supports the JavaBean Validation API (also known as JSR 303; <https://jcp.org/en/jsr/detail?id=303>). This makes it easy to declare validation rules as opposed to explicitly writing declaration logic in your application code.

To apply validation in Spring MVC, you need to

- Add the Spring Validation starter to the build.
- Declare validation rules on the class that is to be validated: specifically, the `Taco` class.

- Specify that validation should be performed in the controller methods that require validation: specifically, the `DesignTacoController`'s `processTaco()` method and the `OrderController`'s `processOrder()` method.
- Modify the form views to display validation errors.

The Validation API offers several annotations that can be placed on properties of domain objects to declare validation rules. Hibernate's implementation of the Validation API adds even more validation annotations. Both can be added to a project by adding the Spring Validation starter to the build. The Validation check box under I/O in the Spring Boot Starter wizard will get the job done, but if you prefer manually editing your build, the following entry in the Maven `pom.xml` file will do the trick:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

Or if you're using Gradle, then this is the dependency you'll need:

```
implementation 'org.springframework.boot:spring-boot-starter-validation'
```

Is the validation starter required?

In earlier versions of Spring Boot, the Spring Validation starter was automatically included with the web starter. Starting with Spring Boot 2.3.0, you'll need to explicitly add it to your build if you intend to apply validation.

With the validation starter in place, let's see how you can apply a few annotations to validate a submitted Taco or `TacoOrder`.

2.3.1 Declaring validation rules

For the Taco class, you want to ensure that the name property isn't empty or null and that the list of selected ingredients has at least one item. The following listing shows an updated Taco class that uses `@NotNull` and `@Size` to declare those validation rules.

Listing 2.11 Adding validation to the Taco domain class

```
package tacos;
import java.util.List;
import javax.validation.constraints.NotNull;
import javax.validation.constraints.Size;
import lombok.Data;

@Data
public class Taco {

    @NotNull
    @Size(min=5, message="Name must be at least 5 characters long")
    private String name;
```

```
@NotNull
@Size(min=1, message="You must choose at least 1 ingredient")
private List<Ingredient> ingredients;

}
```

You'll notice that in addition to requiring that the name property isn't null, you declare that it should have a value that's at least five characters in length.

When it comes to declaring validation on submitted taco orders, you must apply annotations to the `TacoOrder` class. For the address properties, you want to be sure that the user doesn't leave any of the fields blank. For that, you'll use the `@NotBlank` annotation.

Validation of the payment fields, however, is a bit more exotic. You need to ensure not only that the `ccNumber` property isn't empty but also that it contains a value that could be a valid credit card number. The `ccExpiration` property must conform to a format of MM/YY (two-digit month and year), and the `ccCVV` property needs to be a three-digit number. To achieve this kind of validation, you need to use a few other Java-Bean Validation API annotations and borrow a validation annotation from the Hibernate Validator collection of annotations. The following listing shows the changes needed to validate the `TacoOrder` class.

Listing 2.12 Validating order fields

```
package tacos;
import javax.validation.constraints.Digits;
import javax.validation.constraints.NotBlank;
import javax.validation.constraints.Pattern;
import org.hibernate.validator.constraints.CreditCardNumber;
import java.util.List;
import java.util.ArrayList;
import lombok.Data;

@Data
public class TacoOrder {

    @NotBlank(message="Delivery name is required")
    private String deliveryName;

    @NotBlank(message="Street is required")
    private String deliveryStreet;

    @NotBlank(message="City is required")
    private String deliveryCity;

    @NotBlank(message="State is required")
    private String deliveryState;

    @NotBlank(message="Zip code is required")
    private String deliveryZip;
```

```

@CreditCardNumber(message="Not a valid credit card number")
private String ccNumber;

@Pattern(regexp="^(0[1-9]|1[0-2])([\\/])([2-9][0-9])$",
        message="Must be formatted MM/YY")
private String ccExpiration;

@Digits(integer=3, fraction=0, message="Invalid CVV")
private String ccCVV;

private List<Taco> tacos = new ArrayList<>();

public void addTaco(Taco taco) {
    this.tacos.add(taco);
}
}

```

As you can see, the `ccNumber` property is annotated with `@CreditCardNumber`. This annotation declares that the property's value must be a valid credit card number that passes the Luhn algorithm check (<https://creditcardvalidator.org/articles/luhn-algorithm>). This prevents user mistakes and deliberately bad data but doesn't guarantee that the credit card number is actually assigned to an account or that the account can be used for charging.

Unfortunately, there's no ready-made annotation for validating the MM/YY format of the `ccExpiration` property. I've applied the `@Pattern` annotation, providing it with a regular expression that ensures that the property value adheres to the desired format. If you're wondering how to decipher the regular expression, I encourage you to check out the many online regular expression guides, including <http://www.regular-expressions.info/>. Regular expression syntax is a dark art and certainly outside the scope of this book. Finally, we annotate the `ccCVV` property with `@Digits` to ensure that the value contains exactly three numeric digits.

All of the validation annotations include a `message` attribute that defines the message you'll display to the user if the information they enter doesn't meet the requirements of the declared validation rules.

2.3.2 *Performing validation at form binding*

Now that you've declared how a `Taco` and `TacoOrder` should be validated, we need to revisit each of the controllers, specifying that validation should be performed when the forms are POSTed to their respective handler methods.

To validate a submitted `Taco`, you need to add the JavaBean Validation API's `@Valid` annotation to the `Taco` argument of the `DesignTacoController`'s `processTaco()` method, as shown next.

Listing 2.13 Validating a POSTed Taco

```
import javax.validation.Valid;
import org.springframework.validation.Errors;

...

@PostMapping
public String processTaco(
    @Valid Taco taco, Errors errors,
    @ModelAttribute TacoOrder tacoOrder) {

    if (errors.hasErrors()) {
        return "design";
    }

    tacoOrder.addTaco(taco);
    log.info("Processing taco: {}", taco);

    return "redirect:/orders/current";
}
```

The `@Valid` annotation tells Spring MVC to perform validation on the submitted Taco object after it's bound to the submitted form data and before the `processTaco()` method is called. If there are any validation errors, the details of those errors will be captured in an `Errors` object that's passed into `processTaco()`. The first few lines of `processTaco()` consult the `Errors` object, asking its `hasErrors()` method if there are any validation errors. If there are, the method concludes without processing the Taco and returns the "design" view name so that the form is redisplayed.

To perform validation on submitted `TacoOrder` objects, similar changes are also required in the `processOrder()` method of `OrderController`, as shown in the next code listing.

Listing 2.14 Validating a POSTed TacoOrder

```
@PostMapping
public String processOrder(@Valid TacoOrder order, Errors errors,
    SessionStatus sessionStatus) {
    if (errors.hasErrors()) {
        return "orderForm";
    }

    log.info("Order submitted: {}", order);
    sessionStatus.setComplete();

    return "redirect:/";
}
```

In both cases, the method will be allowed to process the submitted data if there are no validation errors. If there are validation errors, the request will be forwarded to the form view to give the user a chance to correct their mistakes.

But how will the user know what mistakes require correction? Unless you call out the errors on the form, the user will be left guessing about how to successfully submit the form.

2.3.3 *Displaying validation errors*

Thymeleaf offers convenient access to the `Errors` object via the `fields` property and with its `th:errors` attribute. For example, to display validation errors on the credit card number field, you can add a `<span>` element that uses these error references to the order form template, as follows.

Listing 2.15 Displaying validation errors

```
<label for="ccNumber">Credit Card #: </label>
  <input type="text" th:field="*{ccNumber}"/>
  <span class="validationError"
    th:if="$#{#fields.hasErrors('ccNumber')}"
    th:errors="*{ccNumber}">CC Num Error</span>
```

Aside from a `class` attribute that can be used to style the error so that it catches the user's attention, the `<span>` element uses a `th:if` attribute to decide whether to display the `<span>`. The `fields` property's `hasErrors()` method checks whether there are any errors in the `ccNumber` field. If so, the `<span>` will be rendered.

The `th:errors` attribute references the `ccNumber` field and, assuming errors exist for that field, it will replace the placeholder content of the `<span>` element with the validation message.

If you were to sprinkle similar `<span>` tags around the order form for the other fields, you might see a form that looks like figure 2.5 when you submit invalid information. The errors indicate that the name, city, and ZIP code fields have been left blank and that all of the payment fields fail to meet the validation criteria.

Now your Taco Cloud controllers not only display and capture input, but they also validate that the information meets some basic validation rules. Let's step back and reconsider the `HomeController` from chapter 1, looking at an alternative implementation.

2.4 *Working with view controllers*

Thus far, you've written three controllers for the Taco Cloud application. Although each controller serves a distinct purpose in the functionality of the application, they all pretty much adhere to the following programming model:

- They're all annotated with `@Controller` to indicate that they're controller classes that should be automatically discovered by Spring component scanning and instantiated as beans in the Spring application context.
- All but `HomeController` are annotated with `@RequestMapping` at the class level to define a baseline request pattern that the controller will handle.

localhost

Order your taco creations!



[Design another taco](#)

Please correct the problems below and resubmit.

Deliver my taco masterpieces to...

Name: Name is required

Street address:

City: City is required

State:

Zip code: Zip code is required

Here's how I'll pay...

Credit Card #: Not a valid credit card number

Expiration: Must be formatted MM/YY

CVV: Invalid CVV

Figure 2.5 Validation errors displayed on the order form

- They all have one or more methods that are annotated with `@GetMapping` or `@PostMapping` to provide specifics on which methods should handle which kinds of requests.

Most of the controllers you'll write will follow that pattern. But when a controller is simple enough that it doesn't populate a model or process input—as is the case with your `HomeController`—there's another way that you can define the controller. Have a

look at the next listing to see how you can declare a view controller—a controller that does nothing but forward the request to a view.

Listing 2.16 Declaring a view controller

```
package tacos.web;

import org.springframework.context.annotation.Configuration;
import
    org.springframework.web.servlet.config.annotation.ViewControllerRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration
public class WebConfig implements WebMvcConfigurer {

    @Override
    public void addViewControllers(ViewControllerRegistry registry) {
        registry.addViewController("/").setViewName("home");
    }
}
```

The most significant thing to notice about `WebConfig` is that it implements the `WebMvcConfigurer` interface. `WebMvcConfigurer` defines several methods for configuring Spring MVC. Even though it's an interface, it provides default implementations of all the methods, so you need to override only the methods you need. In this case, you override `addViewControllers()`.

The `addViewControllers()` method is given a `ViewControllerRegistry` that you can use to register one or more view controllers. Here, you call `addViewController()` on the registry, passing in `/`, which is the path for which your view controller will handle GET requests. That method returns a `ViewControllerRegistration` object, on which you immediately call `setViewName()` to specify `home` as the view that a request for `/` should be forwarded to.

And just like that, you've been able to replace `HomeController` with a few lines in a configuration class. You can now delete `HomeController`, and the application should still behave as it did before. The only other change required is to revisit `HomeControllerTest` from chapter 1, removing the reference to `HomeController` from the `@WebMvcTest` annotation, so that the test class will compile without errors.

Here, you've created a new `WebConfig` configuration class to house the view controller declaration. But any configuration class can implement `WebMvcConfigurer` and override the `addViewController` method. For instance, you could have added the same view controller declaration to the bootstrap `TacoCloudApplication` class like this:

```
@SpringBootApplication
public class TacoCloudApplication implements WebMvcConfigurer {

    public static void main(String[] args) {
        SpringApplication.run(TacoCloudApplication.class, args);
    }
}
```

```

@Override
public void addViewControllers(ViewControllerRegistry registry) {
    registry.addViewController("/").setViewName("home");
}
}

```

By extending an existing configuration class, you can avoid creating a new configuration class, keeping your project artifact count down. But I prefer creating a new configuration class for each kind of configuration (web, data, security, and so on), keeping the application bootstrap configuration clean and simple.

Speaking of view controllers—and more generically, the views that controllers forward requests to—so far you’ve been using Thymeleaf for all of your views. I like Thymeleaf a lot, but maybe you prefer a different template model for your application views. Let’s have a look at Spring’s many supported view options.

2.5 Choosing a view template library

For the most part, your choice of a view template library is a matter of personal taste. Spring is flexible and supports many common templating options. With only a few small exceptions, the template library you choose will itself have no idea that it’s even working with Spring.³

Table 2.2 catalogs the template options supported by Spring Boot autoconfiguration.

Table 2.2 Supported template options

Template	Spring Boot starter dependency
FreeMarker	spring-boot-starter-freemarker
Groovy templates	spring-boot-starter-groovy-templates
JavaServer Pages (JSP)	None (provided by Tomcat or Jetty)
Mustache	spring-boot-starter-mustache
Thymeleaf	spring-boot-starter-thymeleaf

Generally speaking, you select the view template library you want, add it as a dependency in your build, and start writing templates in the `/templates` directory (under the `src/main/resources` directory in a Maven or Gradle project). Spring Boot detects your chosen template library and automatically configures the components required for it to serve views for your Spring MVC controllers.

You’ve already done this with Thymeleaf for the Taco Cloud application. In chapter 1, you selected the Thymeleaf check box when initializing the project. This resulted in Spring Boot’s Thymeleaf starter being included in the `pom.xml` file. When

³ One such exception is Thymeleaf’s Spring Security dialect, which we’ll talk about in chapter 5.

the application starts up, Spring Boot autoconfiguration detects the presence of Thymeleaf and automatically configures the Thymeleaf beans for you. All you had to do was start writing templates in `/templates`.

If you'd rather use a different template library, you simply select it at project initialization or edit your existing project build to include the newly chosen template library.

For example, let's say you wanted to use Mustache instead of Thymeleaf. No problem. Just visit the project `pom.xml` file and replace this

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency>
```

with this:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-mustache</artifactId>
</dependency>
```

Of course, you'd need to make sure that you write all the templates with Mustache syntax instead of Thymeleaf tags. The specifics of working with Mustache (or any of the template language choices) is well outside of the scope of this book, but to give you an idea of what to expect, here's a snippet from a Mustache template that will render one of the ingredient groups in the taco design form:

```
<h3>Designate your wrap:</h3>
{{#wrap}}
<div>
  <input name="ingredients" type="checkbox" value="{{id}}" />
  <span>{{name}}</span><br/>
</div>
{{/wrap}}
```

This is the Mustache equivalent of the Thymeleaf snippet in section 2.1.3. The `{{#wrap}}` block (which concludes with `{{/wrap}}`) iterates through a collection in the request attribute whose key is `wrap` and renders the embedded HTML for each item. The `{{id}}` and `{{name}}` tags reference the `id` and `name` properties of the item (which should be an `Ingredient`).

You'll notice in table 2.2 that JSP doesn't require any special dependency in the build. That's because the servlet container itself (Tomcat by default) implements the JSP specification, thus requiring no further dependencies.

But there's a gotcha if you choose to use JSP. As it turns out, Java servlet containers—including embedded Tomcat and Jetty containers—usually look for JSPs somewhere under `/WEB-INF`. But if you're building your application as an executable JAR

file, there's no way to satisfy that requirement. Therefore, JSP is an option only if you're building your application as a WAR file and deploying it in a traditional servlet container. If you're building an executable JAR file, you must choose Thymeleaf, FreeMarker, or one of the other options in table 2.2.

2.5.1 Caching templates

By default, templates are parsed only once—when they're first used—and the results of that parse are cached for subsequent use. This is a great feature for production, because it prevents redundant template parsing on each request and thus improves performance.

That feature is not so awesome at development time, however. Let's say you fire up your application, hit the taco design page, and decide to make a few changes to it. When you refresh your web browser, you'll still be shown the original version. The only way you can see your changes is to restart the application, which is quite inconvenient.

Fortunately, we have a way to disable caching. All we need to do is set a template-appropriate caching property to false. Table 2.3 lists the caching properties for each of the supported template libraries.

Table 2.3 Properties to enable/disable template caching

Template	Cache-enable property
FreeMarker	<code>spring.freemarker.cache</code>
Groovy templates	<code>spring.groovy.template.cache</code>
Mustache	<code>spring.mustache.cache</code>
Thymeleaf	<code>spring.thymeleaf.cache</code>

By default, all of these properties are set to true to enable caching. You can disable caching for your chosen template engine by setting its cache property to false. For example, to disable Thymeleaf caching, add the following line in `application.properties`:

```
spring.thymeleaf.cache=false
```

The only catch is that you'll want to be sure to remove this line (or set it to true) before you deploy your application to production. One option is to set the property in a profile. (We'll talk about profiles in chapter 6.)

A much simpler option is to use Spring Boot's DevTools, as we opted to do in chapter 1. Among the many helpful bits of development-time help offered by DevTools, it will disable caching for all template libraries but will disable itself (and thus reenable template caching) when your application is deployed.

Summary

- Spring offers a powerful web framework called Spring MVC that can be used to develop the web frontend for a Spring application.
- Spring MVC is annotation-based, enabling the declaration of request-handling methods with annotations such as `@RequestMapping`, `@GetMapping`, and `@PostMapping`.
- Most request-handling methods conclude by returning the logical name of a view, such as a Thymeleaf template, to which the request (along with any model data) is forwarded.
- Spring MVC supports validation through the JavaBean Validation API and implementations of the Validation API such as Hibernate Validator.
- View controllers can be registered with `addViewController` in a `WebMvcConfigurer` class to handle HTTP GET requests for which no model data or processing is required.
- In addition to Thymeleaf, Spring supports a variety of view options, including FreeMarker, Groovy templates, and Mustache.

Working with data

This chapter covers

- Using Spring's `JdbcTemplate`
- Creating Spring Data JDBC repositories
- Declaring JPA repositories with Spring Data

Most applications offer more than just a pretty face. Although the user interface may provide interaction with an application, it's the data it presents and stores that separates applications from static websites.

In the Taco Cloud application, you need to be able to maintain information about ingredients, tacos, and orders. Without a database to store this information, the application wouldn't be able to progress much further than what you developed in chapter 2.

In this chapter, you're going to add data persistence to the Taco Cloud application. You'll start by using Spring support for JDBC (Java Database Connectivity) to eliminate boilerplate code. Then you'll rework the data repositories to work with JPA (Java Persistence API), eliminating even more code.

3.1 *Reading and writing data with JDBC*

For decades, relational databases and SQL have enjoyed their position as the leading choice for data persistence. Even though many alternative database types have emerged in recent years, the relational database is still a top choice for a general-purpose data store and will not likely be usurped from its position any time soon.

When it comes to working with relational data, Java developers have several options. The two most common choices are JDBC and JPA. Spring supports both with abstractions, making working with either JDBC or JPA easier than it would be without Spring. In this section, we'll focus on how Spring supports JDBC, and then we'll look at Spring support for JPA in section 3.2.

Spring JDBC support is rooted in the `JdbcTemplate` class. `JdbcTemplate` provides a means by which developers can perform SQL operations against a relational database without all the ceremony and boilerplate typically required when working with JDBC.

To gain an appreciation of what `JdbcTemplate` does, let's start by looking at an example of how to perform a simple query in Java without `JdbcTemplate`.

Listing 3.1 Querying a database without `JdbcTemplate`

```
@Override
public Optional<Ingredient> findById(String id) {
    Connection connection = null;
    PreparedStatement statement = null;
    ResultSet resultSet = null;
    try {
        connection = dataSource.getConnection();
        statement = connection.prepareStatement(
            "select id, name, type from Ingredient where id=?");
        statement.setString(1, id);
        resultSet = statement.executeQuery();
        Ingredient ingredient = null;
        if(resultSet.next()) {
            ingredient = new Ingredient(
                resultSet.getString("id"),
                resultSet.getString("name"),
                Ingredient.Type.valueOf(resultSet.getString("type")));
        }
        return Optional.of(ingredient);
    } catch (SQLException e) {
        // ??? What should be done here ???
    } finally {
        if (resultSet != null) {
            try {
                resultSet.close();
            } catch (SQLException e) {}
        }
        if (statement != null) {
            try {
                statement.close();
            } catch (SQLException e) {}
        }
    }
}
```

```

        if (connection != null) {
            try {
                connection.close();
            } catch (SQLException e) {}
        }
    }
    return Optional.empty();
}

```

I assure you that somewhere in listing 3.1 are a couple of lines that query the database for ingredients. But I'll bet you had a hard time spotting that query needle in the JDBC haystack. It's surrounded by code that creates a connection, creates a statement, and cleans up by closing the connection, statement, and result set.

To make matters worse, any number of things could go wrong when creating the connection or the statement, or when performing the query. This requires that you catch a `SQLException`, which may or may not be helpful in figuring out what went wrong or how to address the problem.

`SQLException` is a checked exception, which requires handling in a catch block. But the most common problems, such as failure to create a connection to the database or a mistyped query, can't possibly be addressed in a catch block and are likely to be rethrown for handling upstream. In contrast, consider the following method that uses Spring's `JdbcTemplate`.

Listing 3.2 Querying a database with `JdbcTemplate`

```

private JdbcTemplate jdbcTemplate;

public Optional<Ingredient> findById(String id) {
    List<Ingredient> results = jdbcTemplate.query(
        "select id, name, type from Ingredient where id=?",
        this::mapRowToIngredient,
        id);
    return results.size() == 0 ?
        Optional.empty() :
        Optional.of(results.get(0));
}

private Ingredient mapRowToIngredient(ResultSet row, int rowNum)
    throws SQLException {
    return new Ingredient(
        row.getString("id"),
        row.getString("name"),
        Ingredient.Type.valueOf(row.getString("type")));
}

```

The code in listing 3.2 is clearly much simpler than the raw JDBC example in listing 3.1; there aren't any statements or connections being created. And, after the method is finished, there isn't any cleanup of those objects. Finally, there isn't any handling of exceptions that can't properly be handled in a catch block. What's left is code that's focused solely on performing a query (the call to `JdbcTemplate`'s `query()`

method) and mapping the results to an `Ingredient` object (handled by the `mapRowToIngredient()` method).

The code in listing 3.2 is a snippet of what you need to do to use `JdbcTemplate` to persist and read data in the Taco Cloud application. Let's take the next steps necessary to outfit the application with JDBC persistence. We'll start by making a few tweaks to the domain objects.

3.1.1 *Adapting the domain for persistence*

When persisting objects to a database, it's generally a good idea to have one field that uniquely identifies the object. Your `Ingredient` class already has an `id` field, but you need to add `id` fields to both `Taco` and `TacoOrder` as well.

Moreover, it might be useful to know when a `Taco` is created and when a `TacoOrder` is placed. You'll also need to add a field to each object to capture the date and time that the objects are saved. The following listing shows the new `id` and `createdAt` fields needed in the `Taco` class.

Listing 3.3 Adding ID and timestamp fields to the `Taco` class

```
@Data
public class Taco {

    private Long id;

    private Date createdAt = new Date();

    // ...

}
```

Because you use Lombok to automatically generate accessor methods at run time, there's no need to do anything more than declare the `id` and `createdAt` properties. They'll have appropriate getter and setter methods as needed at run time. Similar changes are required in the `TacoOrder` class, as shown here:

```
@Data
public class TacoOrder implements Serializable {

    private static final long serialVersionUID = 1L;

    private Long id;

    private Date placedAt;

    // ...

}
```

Again, Lombok automatically generates the accessor methods, so these are the only changes required in `TacoOrder`. If for some reason you choose not to use Lombok, you'll need to write these methods yourself.

Your domain classes are now ready for persistence. Let's see how to use `JdbcTemplate` to read and write them to a database.

3.1.2 Working with `JdbcTemplate`

Before you can start using `JdbcTemplate`, you need to add it to your project classpath. You can do this easily by adding Spring Boot's JDBC starter dependency to the build as follows:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-jdbc</artifactId>
</dependency>
```

You're also going to need a database where your data will be stored. For development purposes, an embedded database will be just fine. I favor the H2 embedded database, so I've added the following dependency to the build:

```
<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <scope>runtime</scope>
</dependency>
```

By default, the database name is randomly generated. But that makes it hard to determine the database URL if, for some reason, you need to connect to the database using the H2 console (which Spring Boot DevTools enables at <http://localhost:8080/h2-console>). So, it's a good idea to pin down the database name by setting a couple of properties in `application.properties`, as shown next:

```
spring.datasource.generate-unique-name=false
spring.datasource.name=tacocloud
```

Or, if you prefer, rename `application.properties` to `application.yml` and add the properties in YAML format like so:

```
spring:
  datasource:
    generate-unique-name: false
    name: tacocloud
```

The choice between properties file format and YAML format is up to you. Spring Boot is happy to work with either. Given the structure and increased readability of YAML, we'll use YAML for configuration properties throughout the rest of the book.

By setting the `spring.datasource.generate-unique-name` property to `false`, we're telling Spring to not generate a unique random value for the database name. Instead, it should use the value set to the `spring.datasource.name` property. In this case, the database name will be `"tacocloud"`. Consequently, the database URL will be

"jdbc:h2:mem:tacocloud", which you can specify in the JDBC URL for the H2 console connection.

Later, you'll see how to configure the application to use an external database. But for now, let's move on to writing a repository that fetches and saves Ingredient data.

DEFINING JDBC REPOSITORIES

Your Ingredient repository needs to perform the following operations:

- Query for all ingredients into a collection of Ingredient objects
- Query for a single Ingredient by its id
- Save an Ingredient object

The following IngredientRepository interface defines those three operations as method declarations:

```
package tacos.data;

import java.util.Optional;

import tacos.Ingredient;

public interface IngredientRepository {

    Iterable<Ingredient> findAll();

    Optional<Ingredient> findById(String id);

    Ingredient save(Ingredient ingredient);

}
```

Although the interface captures the essence of what you need an ingredient repository to do, you'll still need to write an implementation of IngredientRepository that uses JdbcTemplate to query the database. The code shown next is the first step in writing that implementation.

Listing 3.4 Beginning an ingredient repository with JdbcTemplate

```
package tacos.data;

import java.sql.ResultSet;
import java.sql.SQLException;

import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Repository;

import tacos.Ingredient;

@Repository
public class JdbcIngredientRepository implements IngredientRepository {

    private JdbcTemplate jdbcTemplate;
```

```

public JdbcIngredientRepository(JdbcTemplate jdbcTemplate) {
    this.jdbcTemplate = jdbcTemplate;
}

// ...

}

```

As you can see, `JdbcIngredientRepository` is annotated with `@Repository`. This annotation is one of a handful of stereotype annotations that Spring defines, including `@Controller` and `@Component`. By annotating `JdbcIngredientRepository` with `@Repository`, you declare that it should be automatically discovered by Spring component scanning and instantiated as a bean in the Spring application context.

When Spring creates the `JdbcIngredientRepository` bean, it injects it with `JdbcTemplate`. That's because when there's only one constructor, Spring implicitly applies autowiring of dependencies through that constructor's parameters. If there is more than one constructor, or if you just want autowiring to be explicitly stated, then you can annotate the constructor with `@Autowired` as follows:

```

@Autowired
public JdbcIngredientRepository(JdbcTemplate jdbcTemplate) {
    this.jdbcTemplate = jdbcTemplate;
}

```

The constructor assigns `JdbcTemplate` to an instance variable that will be used in other methods to query and insert into the database. Speaking of those other methods, let's take a look at the implementations of `findAll()` and `findById()`, shown in the code sample.

Listing 3.5 Querying the database with `JdbcTemplate`

```

@Override
public Iterable<Ingredient> findAll() {
    return jdbcTemplate.query(
        "select id, name, type from Ingredient",
        this::mapRowToIngredient);
}

@Override
public Optional<Ingredient> findById(String id) {
    List<Ingredient> results = jdbcTemplate.query(
        "select id, name, type from Ingredient where id=?",
        this::mapRowToIngredient,
        id);
    return results.size() == 0 ?
        Optional.empty() :
        Optional.of(results.get(0));
}

private Ingredient mapRowToIngredient(ResultSet row, int rowNum)
    throws SQLException {

```

```

return new Ingredient(
    row.getString("id"),
    row.getString("name"),
    Ingredient.Type.valueOf(row.getString("type")));
}

```

Both `findAll()` and `findById()` use `JdbcTemplate` in a similar way. The `findAll()` method, expecting to return a collection of objects, uses `JdbcTemplate`'s `query()` method. The `query()` method accepts the SQL for the query as well as an implementation of Spring's `RowMapper` for the purpose of mapping each row in the result set to an object. `query()` also accepts as its final argument(s) a list of any parameters required in the query. But, in this case, there aren't any required parameters.

In contrast, the `findById()` method will need to include a `where` clause in its query to compare the value of the `id` column with the value of the `id` parameter passed into the method. Therefore, the call to `query()` includes, as its final parameter, the `id` parameter. When the query is performed, the `?` will be replaced with this value.

As shown in listing 3.5, the `RowMapper` parameter for both `findAll()` and `findById()` is given as a method reference to the `mapRowToIngredient()` method. Java's method references and lambdas are convenient when working with `JdbcTemplate` as an alternative to an explicit `RowMapper` implementation. If for some reason you want or need an explicit `RowMapper`, then the following implementation of `findById()` shows how to do that:

```

@Override
public Ingredient findById(String id) {
    return jdbcTemplate.queryForObject(
        "select id, name, type from Ingredient where id=?",
        new RowMapper<Ingredient>() {
            public Ingredient mapRow(ResultSet rs, int rowNum)
                throws SQLException {
                return new Ingredient(
                    rs.getString("id"),
                    rs.getString("name"),
                    Ingredient.Type.valueOf(rs.getString("type")));
            }
        }, id);
}

```

Reading data from a database is only part of the story. At some point, data must be written to the database so that it can be read. Let's see about implementing the `save()` method.

INSERTING A ROW

`JdbcTemplate`'s `update()` method can be used for any query that writes or updates data in the database. And, as shown in the following listing, it can be used to insert data into the database.

Listing 3.6 Inserting data with JdbcTemplate

```

@Override
public Ingredient save(Ingredient ingredient) {
    jdbcTemplate.update(
        "insert into Ingredient (id, name, type) values (?, ?, ?)",
        ingredient.getId(),
        ingredient.getName(),
        ingredient.getType().toString());
    return ingredient;
}

```

Because it isn't necessary to map `ResultSet` data to an object, the `update()` method is much simpler than `query()`. It requires only a `String` containing the SQL to perform as well as values to assign to any query parameters. In this case, the query has three parameters, which correspond to the final three parameters of the `save()` method, providing the ingredient's ID, name, and type.

With `JdbcIngredientRepository` complete, you can now inject it into `DesignTacoController` and use it to provide a list of `Ingredient` objects instead of using hardcoded values (as you did in chapter 2). The changes to `DesignTacoController` are shown next.

Listing 3.7 Injecting and using a repository in the controller

```

@Controller
@RequestMapping("/design")
@SessionAttributes("tacoOrder")
public class DesignTacoController {

    private final IngredientRepository ingredientRepo;

    @Autowired
    public DesignTacoController(
        IngredientRepository ingredientRepo) {
        this.ingredientRepo = ingredientRepo;
    }

    @ModelAttribute
    public void addIngredientsToModel(Model model) {
        Iterable<Ingredient> ingredients = ingredientRepo.findAll();
        Type[] types = Ingredient.Type.values();
        for (Type type : types) {
            model.addAttribute(type.toString().toLowerCase(),
                filterByType(ingredients, type));
        }
    }

    // ...
}

```

The `addIngredientsToModel()` method uses the injected `IngredientRepository`'s `findAll()` method to fetch all ingredients from the database. It then filters them into distinct ingredient types before adding them to the model.

Now that we have an `IngredientRepository` from which to fetch `Ingredient` objects, we can also simplify the `IngredientByIdConverter` that we created in chapter 2, replacing its hardcoded `Map` of `Ingredient` objects with a simple call to the `IngredientRepository.findById()` method, as shown next.

Listing 3.8 Simplifying `IngredientByIdConverter`

```
package tacos.web;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.core.convert.converter.Converter;
import org.springframework.stereotype.Component;

import tacos.Ingredient;
import tacos.data.IngredientRepository;

@Component
public class IngredientByIdConverter implements Converter<String, Ingredient> {

    private IngredientRepository ingredientRepo;

    @Autowired
    public IngredientByIdConverter(IngredientRepository ingredientRepo) {
        this.ingredientRepo = ingredientRepo;
    }

    @Override
    public Ingredient convert(String id) {
        return ingredientRepo.findById(id).orElse(null);
    }
}
```

You're almost ready to fire up the application and try out these changes. But before you can start reading data from the `Ingredient` table referenced in the queries, you should probably create that table and populate it with some ingredient data.

3.1.3 Defining a schema and preloading data

Aside from the `Ingredient` table, you're also going to need some tables that hold order and design information. Figure 3.1 illustrates the tables you'll need, as well as the relationships between those tables.

The tables in figure 3.1 serve the following purposes:

- *Taco_Order*—Holds essential order details
- *Taco*—Holds essential information about a taco design
- *Ingredient_Ref*—Contains one or more rows for each row in *Taco*, mapping the taco to the ingredients for that taco
- *Ingredient*—Holds ingredient information

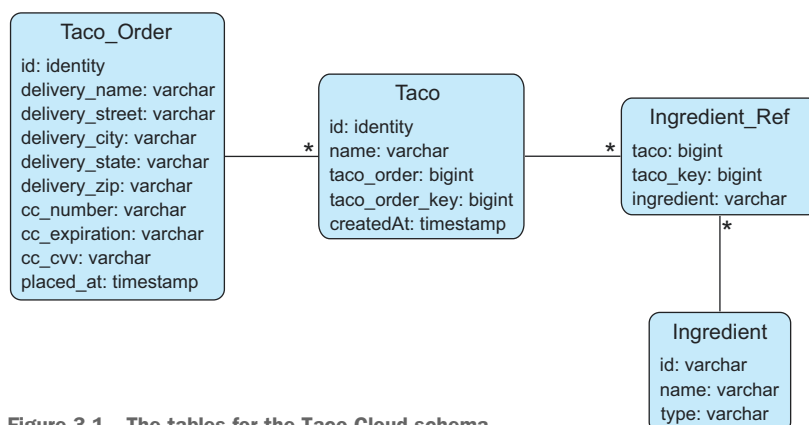


Figure 3.1 The tables for the Taco Cloud schema

In our application, a Taco can't exist outside of the context of a Taco_Order. Thus, Taco_Order and Taco are considered members of an aggregate where Taco_Order is the aggregate root. Ingredient objects, on the other hand, are sole members of their own aggregate and are referenced by Taco by way of Ingredient_Ref.

NOTE Aggregates and aggregate roots are core concepts of *domain-driven design*, a design approach that promotes the idea that the structure and language of software code should match the business domain. Although we're applying a little domain-driven design (DDD) in the Taco Cloud domain objects, there's much more to DDD than aggregates and aggregate roots. For more on this subject, read the seminal work on the subject, *Domain-Driven Design: Tackling Complexity in the Heart of Software* (https://www.dddcommunity.org/book/evans_2003/), by Eric Evans.

The next listing shows the SQL that creates the tables.

Listing 3.9 Defining the Taco Cloud schema

```

create table if not exists Taco_Order (
  id identity,
  delivery_Name varchar(50) not null,
  delivery_Street varchar(50) not null,
  delivery_City varchar(50) not null,
  delivery_State varchar(2) not null,
  delivery_Zip varchar(10) not null,
  cc_number varchar(16) not null,
  cc_expiration varchar(5) not null,
  cc_cvv varchar(3) not null,
  placed_at timestamp not null
);

```

```

create table if not exists Taco (
  id identity,

```

```

    name varchar(50) not null,
    taco_order bigint not null,
    taco_order_key bigint not null,
    created_at timestamp not null
);

create table if not exists Ingredient_Ref (
    ingredient varchar(4) not null,
    taco bigint not null,
    taco_key bigint not null
);

create table if not exists Ingredient (
    id varchar(4) not null,
    name varchar(25) not null,
    type varchar(10) not null
);

alter table Taco
    add foreign key (taco_order) references Taco_Order(id);
alter table Ingredient_Ref
    add foreign key (ingredient) references Ingredient(id);

```

The big question is where to put this schema definition. As it turns out, Spring Boot answers that question.

If there's a file named `schema.sql` in the root of the application's classpath, then the SQL in that file will be executed against the database when the application starts. Therefore, you should place the contents of listing 3.8 in your project as a file named `schema.sql` in the `src/main/resources` folder.

You also need to preload the database with some ingredient data. Fortunately, Spring Boot will also execute a file named `data.sql` from the root of the classpath when the application starts. Therefore, you can load the database with ingredient data using the insert statements in the next listing, placed in `src/main/resources/data.sql`.

Listing 3.10 Preloading the database with `data.sql`

```

delete from Ingredient_Ref;
delete from Taco;
delete from Taco_Order;

delete from Ingredient;
insert into Ingredient (id, name, type)
    values ('FLTO', 'Flour Tortilla', 'WRAP');
insert into Ingredient (id, name, type)
    values ('COTO', 'Corn Tortilla', 'WRAP');
insert into Ingredient (id, name, type)
    values ('GRBF', 'Ground Beef', 'PROTEIN');
insert into Ingredient (id, name, type)
    values ('CARN', 'Carnitas', 'PROTEIN');

```

```

insert into Ingredient (id, name, type)
    values ('TMTO', 'Diced Tomatoes', 'VEGGIES');
insert into Ingredient (id, name, type)
    values ('LETC', 'Lettuce', 'VEGGIES');
insert into Ingredient (id, name, type)
    values ('CHED', 'Cheddar', 'CHEESE');
insert into Ingredient (id, name, type)
    values ('JACK', 'Monterrey Jack', 'CHEESE');
insert into Ingredient (id, name, type)
    values ('SLSA', 'Salsa', 'SAUCE');
insert into Ingredient (id, name, type)
    values ('SRCR', 'Sour Cream', 'SAUCE');

```

Even though you’ve only developed a repository for ingredient data, you can fire up the Taco Cloud application at this point and visit the design page to see `JdbcIngredientRepository` in action. Go ahead ... give it a try. When you get back, you’ll write the repositories for persisting Taco and `TacoOrder` data.

3.1.4 Inserting data

You’ve already had a glimpse into how to use `JdbcTemplate` to write data to the database. The `save()` method in `JdbcIngredientRepository` used the `update()` method of `JdbcTemplate` to save `Ingredient` objects to the database.

Although that was a good first example, it was perhaps a bit too simple. As you’ll soon see, saving data can be more involved than what `JdbcIngredientRepository` needed.

In our design, `TacoOrder` and `Taco` are part of an aggregate in which `TacoOrder` is the aggregate root. In other words, `Taco` objects don’t exist outside of the context of a `TacoOrder`. So, for now, we only need to define a repository to persist `TacoOrder` objects and, in turn, `Taco` objects along with them. Such a repository can be defined in a `OrderRepository` interface like this:

```

package tacos.data;

import java.util.Optional;

import tacos.TacoOrder;

public interface OrderRepository {

    TacoOrder save(TacoOrder order);

}

```

Seems simple enough, right? Not so quick. When you save a `TacoOrder`, you also must save the `Taco` objects that go with it. And when you save the `Taco` objects, you’ll also need to save an object that represents the link between the `Taco` and each `Ingredient` that makes up the taco. The `IngredientRef` class defines that linking between `Taco` and `Ingredient` as follows:

```

package tacos;

import lombok.Data;

@Data
public class IngredientRef {

    private final String ingredient;

}

```

Suffice it to say that the `save()` method will be a bit more interesting than the corresponding method you created earlier for saving a humble `Ingredient` object.

Another thing that the `save()` method will need to do is determine what ID is assigned to the order once it has been saved. Per the schema, the `id` property on the `Taco_Order` table is an identity, meaning that the database will determine the value automatically. But if the database determines the value for you, then you will need to know what that value is so that it can be returned in the `TacoOrder` object returned from the `save()` method. Fortunately, Spring offers a helpful `GeneratedKeyHolder` type that can help with that. But it involves working with a prepared statement, as shown in the following implementation of the `save()` method:

```

package tacos.data;

import java.sql.Types;
import java.util.Arrays;
import java.util.Date;
import java.util.List;
import java.util.Optional;

import org.springframework.asm.Type;
import org.springframework.jdbc.core.JdbcOperations;
import org.springframework.jdbc.core.PreparedStatementCreator;
import org.springframework.jdbc.core.PreparedStatementCreatorFactory;
import org.springframework.jdbc.support.GeneratedKeyHolder;
import org.springframework.stereotype.Repository;
import org.springframework.transaction.annotation.Transactional;

import tacos.IngredientRef;
import tacos.Taco;
import tacos.TacoOrder;

@Repository
public class JdbcOrderRepository implements OrderRepository {

    private JdbcOperations jdbcOperations;

    public JdbcOrderRepository(JdbcOperations jdbcOperations) {
        this.jdbcOperations = jdbcOperations;
    }
}

```

```

@Override
@Transactional
public TacoOrder save(TacoOrder order) {
    PreparedStatementCreatorFactory pscf =
        new PreparedStatementCreatorFactory(
            "insert into Taco_Order "
            + "(delivery_name, delivery_street, delivery_city, "
            + "delivery_state, delivery_zip, cc_number, "
            + "cc_expiration, cc_cvv, placed_at) "
            + "values (?, ?, ?, ?, ?, ?, ?, ?, ?)",
            Types.VARCHAR, Types.VARCHAR, Types.VARCHAR,
            Types.VARCHAR, Types.VARCHAR, Types.VARCHAR,
            Types.VARCHAR, Types.VARCHAR, Types.TIMESTAMP
        );
    pscf.setReturnGeneratedKeys(true);

    order.setPlacedAt(new Date());
    PreparedStatementCreator psc =
        pscf.newPreparedStatementCreator(
            Arrays.asList(
                order.getDeliveryName(),
                order.getDeliveryStreet(),
                order.getDeliveryCity(),
                order.getDeliveryState(),
                order.getDeliveryZip(),
                order.getCcNumber(),
                order.getCcExpiration(),
                order.getCcCVV(),
                order.getPlacedAt()
            ));

    GeneratedKeyHolder keyHolder = new GeneratedKeyHolder();
    jdbcOperations.update(psc, keyHolder);
    long orderId = keyHolder.getKey().longValue();
    order.setId(orderId);

    List<Taco> tacos = order.getTacos();
    int i=0;
    for (Taco taco : tacos) {
        saveTaco(orderId, i++, taco);
    }

    return order;
}
}

```

There appears to be a lot going on in the `save()` method, but we can break it down into only a handful of significant steps. First, you create a `PreparedStatementCreatorFactory` that describes the insert query along with the types of the query's input fields. Because you'll later need to fetch the saved order's ID, you also will need to call `setReturnGeneratedKeys(true)`.

After defining the `PreparedStatementCreatorFactory`, you use it to create a `PreparedStatementCreator`, passing in the values from the `TacoOrder` object that will

be persisted. The last field given to the `PreparedStatementCreator` is the date that the order is created, which you'll also need to set on the `TacoOrder` object itself so that the returned `TacoOrder` will have that information available.

Now that you have a `PreparedStatementCreator` in hand, you're ready to actually save the order data by calling the `update()` method on `JdbcTemplate`, passing in the `PreparedStatementCreator` and a `GeneratedKeyHolder`. After the order data has been saved, the `GeneratedKeyHolder` will contain the value of the `id` field as assigned by the database and should be copied into the `TacoOrder` object's `id` property.

At this point, the order has been saved, but you need to also save the `Taco` objects associated with the order. You can do that by calling `saveTaco()` for each `Taco` in the order.

The `saveTaco()` method is quite similar to the `save()` method, as you can see here:

```
private long saveTaco(Long orderId, int orderKey, Taco taco) {
    taco.setCreatedAt(new Date());
    PreparedStatementCreatorFactory pscf =
        new PreparedStatementCreatorFactory(
            "insert into Taco "
            + "(name, created_at, taco_order, taco_order_key) "
            + "values (?, ?, ?, ?)",
            Types.VARCHAR, Types.TIMESTAMP, Type.LONG, Type.LONG
        );
    pscf.setReturnGeneratedKeys(true);

    PreparedStatementCreator psc =
        pscf.newPreparedStatementCreator(
            Arrays.asList(
                taco.getName(),
                taco.getCreatedAt(),
                orderId,
                orderKey
            )
        );

    GeneratedKeyHolder keyHolder = new GeneratedKeyHolder();
    jdbcOperations.update(psc, keyHolder);
    long tacoId = keyHolder.getKey().longValue();
    taco.setId(tacoId);

    saveIngredientRefs(tacoId, taco.getIngredients());

    return tacoId;
}
```

Step by step, `saveTaco()` mirrors the structure of `save()`, albeit for `Taco` data instead of `TacoOrder` data. In the end, it makes a call to `saveIngredientRefs()` to create a row in the `Ingredient_Ref` table to link the `Taco` row to an `Ingredient` row. The `saveIngredientRefs()` method looks like this:

```
private void saveIngredientRefs(
    long tacoId, List<IngredientRef> ingredientRefs) {
```

```

int key = 0;
for (IngredientRef ingredientRef : ingredientRefs) {
    jdbcOperations.update(
        "insert into Ingredient_Ref (ingredient, taco, taco_key) "
        + "values (?, ?, ?)",
        ingredientRef.getIngredient(), tacoId, key++);
}
}

```

Thankfully, the `saveIngredientRefs()` method is much simpler. It cycles through a list of `Ingredient` objects, saving each into the `Ingredient_Ref` table. It also has a local key variable that is used as an index to ensure that the ordering of the ingredients stays intact.

All that's left to do with `OrderRepository` is to inject it into `OrderController` and use it when saving an order. The following listing shows the changes necessary for injecting the repository.

Listing 3.11 Injecting and using `OrderRepository`

```

package tacos.web;
import javax.validation.Valid;

import org.springframework.stereotype.Controller;
import org.springframework.validation.Errors;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.SessionAttributes;
import org.springframework.web.bind.support.SessionStatus;

import tacos.TacoOrder;
import tacos.data.OrderRepository;

@Controller
@RequestMapping("/orders")
@SessionAttributes("tacoOrder")
public class OrderController {

    private OrderRepository orderRepo;

    public OrderController(OrderRepository orderRepo) {
        this.orderRepo = orderRepo;
    }

    // ...

    @PostMapping
    public String processOrder(@Valid TacoOrder order, Errors errors,
        SessionStatus sessionStatus) {
        if (errors.hasErrors()) {
            return "orderForm";
        }
    }
}

```

```
        orderRepo.save(order);  
        sessionStatus.setComplete();  
  
        return "redirect:/";  
    }  
}
```

As you can see, the constructor takes an `OrderRepository` as a parameter and assigns it to an instance variable that it will use in the `processOrder()` method. Speaking of the `processOrder()` method, it has been changed to call the `save()` method on the `OrderRepository` instead of logging the `TacoOrder` object.

Spring's `JdbcTemplate` makes working with relational databases significantly simpler than with plain vanilla JDBC. But even with `JdbcTemplate`, some persistence tasks are still challenging, especially when persisting nested domain objects in an aggregate. If only there were a way to work with JDBC that was even simpler.

Let's have a look at Spring Data JDBC, which makes working with JDBC insanely easy—even when persisting aggregates.

3.2 *Working with Spring Data JDBC*

The Spring Data project is a rather large umbrella project comprising several subprojects, most of which are focused on data persistence with a variety of different database types. A few of the most popular Spring Data projects include these:

- *Spring Data JDBC*—JDBC persistence against a relational database
- *Spring Data JPA*—JPA persistence against a relational database
- *Spring Data MongoDB*—Persistence to a Mongo document database
- *Spring Data Neo4j*—Persistence to a Neo4j graph database
- *Spring Data Redis*—Persistence to a Redis key-value store
- *Spring Data Cassandra*—Persistence to a Cassandra column store database

One of the most interesting and useful features provided by Spring Data for all of these projects is the ability to automatically create repositories, based on a repository specification interface. Consequently, persistence with Spring Data projects has little or no persistence logic and involves writing only one or more repository interfaces.

Let's see how to apply Spring Data JDBC to our project to simplify data persistence with JDBC. First, you'll need to add Spring Data JDBC to the project build.

3.2.1 *Adding Spring Data JDBC to the build*

Spring Data JDBC is available as a starter dependency for Spring Boot apps. When added to the project's `pom.xml` file, the starter dependency looks like the following code snippet.

Listing 3.12 Adding the Spring Data JDBC dependency to the build

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jdbc</artifactId>
</dependency>
```

You will no longer need the JDBC starter that gave us `JdbcTemplate`, so you can remove the starter that looks like this:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-jdbc</artifactId>
</dependency>
```

You'll still need a database, however, so don't remove the H2 dependency.

3.2.2 Defining repository interfaces

Fortunately, we've already created `IngredientRepository` and `OrderRepository`, so much of the work in defining our repositories is already done. But we'll need to make a subtle change to them in order to use them with Spring Data JDBC.

Spring Data will automatically generate implementations for our repository interfaces at run time. But it will do that only for interfaces that extend one of the repository interfaces provided by Spring Data. At the very least, our repository interfaces will need to extend `Repository` so that Spring Data knows to create the implementation automatically. For example, here's how you might write `IngredientRepository` such that it extends `Repository`:

```
package tacos.data;
import java.util.Optional;
import org.springframework.data.repository.Repository;
import tacos.Ingredient;

public interface IngredientRepository
    extends Repository<Ingredient, String> {

    Iterable<Ingredient> findAll();

    Optional<Ingredient> findById(String id);

    Ingredient save(Ingredient ingredient);

}
```

As you can see, the `Repository` interface is parameterized. The first parameter is the type of the object to be persisted by this repository—in this case, `Ingredient`. The second parameter is the type of the persisted object's ID field. For `Ingredient`, that's `String`.

Although `IngredientRepository` will work as shown here by extending `Repository`, Spring Data also offers `CrudRepository` as a base interface for common operations, including the three methods we've defined in `IngredientRepository`. So, instead of extending `Repository`, it's often easier to extend `CrudRepository`, as shown next.

Listing 3.13 Defining a repository interface for persisting ingredients

```
package tacos.data;

import org.springframework.data.repository.CrudRepository;

import tacos.Ingredient;

public interface IngredientRepository
    extends CrudRepository<Ingredient, String> {

}
```

Similarly, our `OrderRepository` can extend `CrudRepository` as shown in the next listing.

Listing 3.14 Defining a repository interface for persisting taco orders

```
package tacos.data;

import org.springframework.data.repository.CrudRepository;

import tacos.TacoOrder;

public interface OrderRepository
    extends CrudRepository<TacoOrder, Long> {

}
```

In both cases, because `CrudRepository` already defines the methods you need, there's no need to explicitly define them in the `IngredientRepository` and `OrderRepository` interfaces.

And now you have your two repositories. You might be thinking that you need to write the implementations for both repositories, including the dozen methods defined in `CrudRepository`. But that's the good news about Spring Data—there's no need to write an implementation! When the application starts, Spring Data automatically generates an implementation on the fly. This means the repositories are ready to use from the get-go. Just inject them into the controllers and you're done.

What's more, because Spring Data automatically creates implementations of these interfaces at run time, you no longer need the explicit implementations in `JdbcIngredientRepository` and `JdbcOrderRepository`. You can delete those two classes and never look back!

3.2.3 Annotating the domain for persistence

The only other thing we'll need to do is annotate our domain classes so that Spring Data JDBC will know how to persist them. Generally speaking, this means annotating the identity properties with `@Id`—so that Spring Data will know which field represents the object's identity—and optionally annotating the class with `@Table`.

For example, the `TacoOrder` class might be annotated with `@Table` and `@Id` as shown in the following code.

Listing 3.15 Preparing the `Taco` class for persistence

```
package tacos;
import java.io.Serializable;
import java.util.ArrayList;
import java.util.Date;
import java.util.List;

import javax.validation.constraints.Digits;
import javax.validation.constraints.NotBlank;
import javax.validation.constraints.Pattern;

import org.hibernate.validator.constraints.CreditCardNumber;
import org.springframework.data.annotation.Id;
import org.springframework.data.relational.core.mapping.Table;

import lombok.Data;

@Data
@Table
public class TacoOrder implements Serializable {

    private static final long serialVersionUID = 1L;

    @Id
    private Long id;

    // ...
}
```

The `@Table` annotation is completely optional. By default, the object is mapped to a table based on the domain class name. In this case, `TacoOrder` is mapped to a table named `"Taco_Order"`. If that's fine for you, then you can leave the `@Table` annotation off completely or use it without parameters. But if you'd prefer to map it to a different table name, then you can specify the table name as a parameter to `@Table` like this:

```
@Table("Taco_Cloud_Order")
public class TacoOrder {
    ...
}
```

As shown here, `TacoOrder` will be mapped to a table named `"Taco_Cloud_Order"`.

As for the `@Id` annotation, it designates the `id` property as being the identity for a `TacoOrder`. All other properties in `TacoOrder` will be mapped automatically to columns based on their property names. For example, the `deliveryName` property will be automatically mapped to the column named `delivery_name`. But if you want to explicitly define the column name mapping, you could annotate the property with `@Column` like this:

```
@Column("customer_name")
@NotBlank(message="Delivery name is required")
private String deliveryName;
```

In this case, `@Column` is specifying that the `deliveryName` property will be mapped to the column whose name is `customer_name`.

You'll also need to apply `@Table` and `@Id` to the other domain classes. This includes `@Ingredient`.

Listing 3.16 Preparing the `Ingredient` class for persistence

```
package tacos;

import org.springframework.data.annotation.Id;
import org.springframework.data.domain.Persistable;
import org.springframework.data.relational.core.mapping.Table;

import lombok.AccessLevel;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@Table
@AllArgsConstructor
@NoArgsConstructor(access=AccessLevel.PRIVATE, force=true)
public class Ingredient implements Persistable<String> {

    @Id
    private String id;

    // ...

}
```

...and `Taco`.

Listing 3.17 Preparing the `Taco` class for persistence

```
package tacos;

import java.util.ArrayList;
import java.util.Date;
import java.util.List;
```

```
import javax.validation.constraints.NotNull;
import javax.validation.constraints.Size;

import org.springframework.data.annotation.Id;
import org.springframework.data.relational.core.mapping.Table;

import lombok.Data;

@Data
@Table
public class Taco {

    @Id
    private Long id;

    // ...

}
```

As for `IngredientRef`, it will be mapped automatically to the table whose name is "Ingredient_Ref", which is perfect for our application. You can annotate it with `@Table` if you want, but it's not necessary. And the "Ingredient_Ref" table has no identity column, so there is no need to annotate anything in `IngredientRef` with `@Id`.

With these small changes, not to mention the complete removal of the `JdbcIngredientRepository` and `JdbcOrderRepository` classes, you now have a lot less persistence code. Even so, the repository implementations that are generated at runtime by Spring Data still do everything that the repositories using `JdbcTemplate` did. In fact, they have potential for doing even more, because the two repository interfaces extend `CrudRepository`, which offers a dozen or so operations for creating, reading, updating, and deleting objects.

3.2.4 Preloading data with `CommandLineRunner`

When working with `JdbcTemplate`, we preloaded the `Ingredient` data at application startup using `data.sql`, which was executed against the database when the data source bean was created. That same approach will work with Spring Data JDBO. In fact, it will work with any persistence mechanism for which the backing database is a relational database. But let's see another way of populating a database at startup that offers a bit more flexibility.

Spring Boot offers two useful interfaces for executing logic when an application starts up: `CommandLineRunner` and `ApplicationRunner`. These two interfaces are quite similar. Both are functional interfaces that require that a single `run()` method be implemented. When the application starts up, any beans in the application context that implement `CommandLineRunner` or `ApplicationRunner` will have their `run()` methods invoked after the application context and all beans are wired up, but before anything else happens. This provides a convenient place for data to be loaded into the database.

Because both `CommandLineRunner` and `ApplicationRunner` are functional interfaces, they can easily be declared as beans in a configuration class using a `@Bean`-annotated

method that returns a lambda function. For example, here's how you might create a data-loading `CommandLineRunner` bean:

```
@Bean
public CommandLineRunner dataLoader(IngredientRepository repo) {
    return args -> {
        repo.save(new Ingredient("FLTO", "Flour Tortilla", Type.WRAP));
        repo.save(new Ingredient("COTO", "Corn Tortilla", Type.WRAP));
        repo.save(new Ingredient("GRBF", "Ground Beef", Type.PROTEIN));
        repo.save(new Ingredient("CARN", "Carnitas", Type.PROTEIN));
        repo.save(new Ingredient("TMTO", "Diced Tomatoes", Type.VEGGIES));
        repo.save(new Ingredient("LETC", "Lettuce", Type.VEGGIES));
        repo.save(new Ingredient("CHED", "Cheddar", Type.CHEESE));
        repo.save(new Ingredient("JACK", "Monterrey Jack", Type.CHEESE));
        repo.save(new Ingredient("SLSA", "Salsa", Type.SAUCE));
        repo.save(new Ingredient("SRCR", "Sour Cream", Type.SAUCE));
    };
}
```

Here, the `IngredientRepository` is injected into the bean method and used within the lambda to create `Ingredient` objects. The `run()` method of `CommandLineRunner` accepts a single parameter that is a `String` vararg containing all of the command-line arguments for the running application. We don't need those to load ingredients into the database, so the `args` parameter is ignored.

Alternatively, we could have defined the data-loader bean as a lambda implementation of `ApplicationRunner` like this:

```
@Bean
public ApplicationRunner dataLoader(IngredientRepository repo) {
    return args -> {
        repo.save(new Ingredient("FLTO", "Flour Tortilla", Type.WRAP));
        repo.save(new Ingredient("COTO", "Corn Tortilla", Type.WRAP));
        repo.save(new Ingredient("GRBF", "Ground Beef", Type.PROTEIN));
        repo.save(new Ingredient("CARN", "Carnitas", Type.PROTEIN));
        repo.save(new Ingredient("TMTO", "Diced Tomatoes", Type.VEGGIES));
        repo.save(new Ingredient("LETC", "Lettuce", Type.VEGGIES));
        repo.save(new Ingredient("CHED", "Cheddar", Type.CHEESE));
        repo.save(new Ingredient("JACK", "Monterrey Jack", Type.CHEESE));
        repo.save(new Ingredient("SLSA", "Salsa", Type.SAUCE));
        repo.save(new Ingredient("SRCR", "Sour Cream", Type.SAUCE));
    };
}
```

The key difference between `CommandLineRunner` and `ApplicationRunner` is in the parameter passed to the respective `run()` methods. `CommandLineRunner` accepts a `String` vararg, which is a raw representation of arguments passed on the command line. But `ApplicationRunner` accepts an `ApplicationArguments` parameter that offers methods for accessing the arguments as parsed components of the command line.

For example, suppose that we want our application to accept a command line with arguments such as `--version 1.2.3` and need to consider that argument in our

loader bean. If using a `CommandLineRunner`, we'd need to search the array for “-version” and then take the very next value from the array. But with `ApplicationRunner`, we can query the given `ApplicationArguments` for the “-version” argument like this:

```
public ApplicationRunner dataLoader(IngredientRepository repo) {
    return args -> {
        List<String> version = args.getOptionValues("version");
        ...
    };
}
```

The `getOptionValues()` method returns a `List<String>` to allow for the option argument to be specified multiple times.

In the case of either `CommandLineRunner` or `ApplicationRunner`, however, we don't need command-line arguments to load data. So the `args` parameter is ignored in our data-loader bean.

What's nice about using `CommandLineRunner` or `ApplicationRunner` to do an initial data load is that they are using the repositories to create the persisted objects instead of a SQL script. This means that they'll work equally well for relational databases and nonrelational databases. This will come in handy in the next chapter when we see how to use Spring Data to persist to nonrelational databases.

But before we do that, let's have a look at another Spring Data project for persisting data in relational databases: Spring Data JPA.

3.3 Persisting data with Spring Data JPA

Whereas Spring Data JDBC makes easy work of persisting data, the Java Persistence API (JPA) is another popular option for working with data in a relational database. Spring Data JPA offers an approach to persistence with JPA similar to what Spring Data JDBC gave us for JDBC.

To see how Spring Data works, you're going to start over, replacing the JDBC-based repositories from earlier in this chapter with repositories created by Spring Data JPA. But first, you need to add Spring Data JPA to the project build.

3.3.1 Adding Spring Data JPA to the project

Spring Data JPA is available to Spring Boot applications with the JPA starter. This starter dependency not only brings in Spring Data JPA but also transitively includes Hibernate as the JPA implementation, as shown here:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
```

If you want to use a different JPA implementation, then you'll need to, at least, exclude the Hibernate dependency and include the JPA library of your choice. For

example, to use EclipseLink instead of Hibernate, you'll need to alter the build as follows:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
  <exclusions>
    <exclusion>
      <groupId>org.hibernate</groupId>
      <artifactId>hibernate-core</artifactId>
    </exclusion>
  </exclusions>
</dependency>
<dependency>
  <groupId>org.eclipse.persistence</groupId>
  <artifactId>org.eclipse.persistence.jpa</artifactId>
  <version>2.7.6</version>
</dependency>
```

Note that there may be other changes required, depending on your choice of JPA implementation. Consult the documentation of your chosen JPA implementation for details. Now let's revisit your domain objects and annotate them for JPA persistence.

3.3.2 *Annotating the domain as entities*

As you've already seen with Spring Data JDBC, Spring Data does some amazing things when it comes to creating repositories. But unfortunately, it doesn't help much when it comes to annotating your domain objects with JPA mapping annotations. You'll need to open up the `Ingredient`, `Taco`, and `TacoOrder` classes and throw in a few annotations. First up is the `Ingredient` class, shown next.

Listing 3.18 Annotating `Ingredient` for JPA persistence

```
package tacos;

import javax.persistence.Entity;
import javax.persistence.Id;

import lombok.AccessLevel;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@Entity
@AllArgsConstructor
@NoArgsConstructor(access=AccessLevel.PRIVATE, force=true)
public class Ingredient {

    @Id
    private String id;
    private String name;
    private Type type;
```

```
public enum Type {  
    WRAP, PROTEIN, VEGGIES, CHEESE, SAUCE  
}  
  
}
```

To declare this as a JPA entity, `Ingredient` must be annotated with `@Entity`. And its `id` property must be annotated with `@Id` to designate it as the property that will uniquely identify the entity in the database. Note that this `@Id` annotation is the JPA variety from the `javax.persistence` package, as opposed to the `@Id` provided by Spring Data in the `org.springframework.data.annotation` package.

Also note that we no longer need the `@Table` annotation or need to implement `Persistable`. Although we could still use `@Table` here, it is unnecessary when working with JPA and defaults to the name of the class ("`Ingredient`", in this case). As for `Persistable`, it was only necessary with Spring Data JDBC to determine whether or not an entity was to be created new, or to update an existing entity; JPA sorts that out automatically.

In addition to the JPA-specific annotations, you'll also note that you've added a `@NoArgsConstructor` annotation at the class level. JPA requires that entities have a no-arguments constructor, so Lombok's `@NoArgsConstructor` does that for you. You don't want to be able to use it, though, so you make it private by setting the access attribute to `AccessLevel.PRIVATE`. And because you must set final properties, you also set the force attribute to true, which results in the Lombok-generated constructor setting them to a default value of null, 0, or false, depending on the property type.

You also will add an `@AllArgsConstructor` to make it easy to create an `Ingredient` object with all properties initialized.

You also need a `@RequiredArgsConstructor`. The `@Data` annotation implicitly adds a required arguments constructor, but when a `@NoArgsConstructor` is used, that constructor is removed. An explicit `@RequiredArgsConstructor` ensures that you'll still have a required arguments constructor, in addition to the private no-arguments constructor.

Now let's move on to the `Taco` class and see how to annotate it as a JPA entity.

Listing 3.19 Annotating Taco as an entity

```
package tacos;  
import java.util.ArrayList;  
import java.util.Date;  
import java.util.List;  
  
import javax.persistence.Entity;  
import javax.persistence.GeneratedValue;  
import javax.persistence.GenerationType;  
import javax.persistence.Id;  
import javax.persistence.ManyToMany;
```

```

import javax.validation.constraints.NotNull;
import javax.validation.constraints.Size;

import lombok.Data;

@Data
@Entity
public class Taco {

    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private Long id;

    @NotNull
    @Size(min=5, message="Name must be at least 5 characters long")
    private String name;

    private Date createdAt = new Date();

    @Size(min=1, message="You must choose at least 1 ingredient")
    @ManyToMany()
    private List<Ingredient> ingredients = new ArrayList<>();

    public void addIngredient(Ingredient ingredient) {
        this.ingredients.add(ingredient);
    }
}

```

As with `Ingredient`, the `Taco` class is now annotated with `@Entity` and has its `id` property annotated with `@Id`. Because you're relying on the database to automatically generate the ID value, you also annotate the `id` property with `@GeneratedValue`, specifying a strategy of `AUTO`.

To declare the relationship between a `Taco` and its associated `Ingredient` list, you annotate `ingredients` with `@ManyToMany`. A `Taco` can have many `Ingredient` objects, and an `Ingredient` can be a part of many `Tacos`.

Finally, let's annotate the `TacoOrder` object as an entity. The next listing shows the new `TacoOrder` class.

Listing 3.20 Annotating `TacoOrder` as a JPA entity

```

package tacos;
import java.io.Serializable;
import java.util.ArrayList;
import java.util.Date;
import java.util.List;

import javax.persistence.CascadeType;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;

```

```

import javax.persistence.OneToMany;
import javax.validation.constraints.Digits;
import javax.validation.constraints.NotBlank;
import javax.validation.constraints.Pattern;

import org.hibernate.validator.constraints.CreditCardNumber;

import lombok.Data;

@Data
@Entity
public class TacoOrder implements Serializable {

    private static final long serialVersionUID = 1L;

    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private Long id;

    private Date placedAt = new Date();

    ...

    @OneToMany(cascade = CascadeType.ALL)
    private List<Taco> tacos = new ArrayList<>();

    public void addTaco(Taco taco) {
        this.tacos.add(taco);
    }
}

```

As you can see, the changes to `TacoOrder` closely mirror the changes to `Taco`. One significant thing worth noting is that the relationship to the list of `Taco` objects is annotated with `@OneToMany`, indicating that the tacos are all specific to this one order. Moreover, the cascade attribute is set to `CascadeType.ALL` so that if the order is deleted, its related tacos will also be deleted.

3.3.3 Declaring JPA repositories

When you created the `JdbcTemplate` versions of the repositories, you explicitly declared the methods you wanted the repository to provide. But with Spring Data JDBC, you were able to dismiss the explicit implementation classes and instead extend the `CrudRepository` interface. As it turns out, `CrudRepository` works equally well for Spring Data JPA. For example, here's the new `IngredientRepository` interface:

```

package tacos.data;

import org.springframework.data.repository.CrudRepository;

import tacos.Ingredient;

```

```
public interface IngredientRepository
    extends CrudRepository<Ingredient, String> {

}
```

In fact, the `IngredientRepository` interface we'll use with Spring Data JPA is identical to the one we defined for use with Spring Data JDBC. The `CrudRepository` interface is commonly used across many of Spring Data's projects, regardless of the underlying persistence mechanism. Similarly, you can define `OrderRepository` for the Spring Data JPA the same as it was for Spring Data JDBC, as follows:

```
package tacos.data;

import org.springframework.data.repository.CrudRepository;

import tacos.TacoOrder;

public interface OrderRepository
    extends CrudRepository<TacoOrder, Long> {

}
```

The methods provided by `CrudRepository` are great for general-purpose persistence of entities. But what if you have some requirements beyond basic persistence? Let's see how to customize the repositories to perform queries unique to your domain.

3.3.4 *Customizing repositories*

Imagine that in addition to the basic CRUD operations provided by `CrudRepository`, you also need to fetch all the orders delivered to a given ZIP code. As it turns out, this can easily be addressed by adding the following method declaration to `OrderRepository`:

```
List<TacoOrder> findByDeliveryZip(String deliveryZip);
```

When generating the repository implementation, Spring Data examines each method in the repository interface, parses the method name, and attempts to understand the method's purpose in the context of the persisted object (a `TacoOrder`, in this case). In essence, Spring Data defines a sort of miniature domain-specific language (DSL), where persistence details are expressed in repository method signatures.

Spring Data knows that this method is intended to find `Orders`, because you've parameterized `CrudRepository` with `TacoOrder`. The method name, `findByDeliveryZip()`, makes it clear that this method should find all `TacoOrder` entities by matching their `deliveryZip` property with the value passed in as a parameter to the method.

The `findByDeliveryZip()` method is simple enough, but Spring Data can handle even more interesting method names as well. Repository methods are composed

of a verb, an optional subject, the word *By*, and a predicate. In the case of `findByDeliveryZip()`, the verb is `find` and the predicate is `DeliveryZip`; the subject isn't specified and is implied to be a `TacoOrder`.

Let's consider another, more complex example. Suppose that you need to query for all orders delivered to a given ZIP code within a given date range. In that case, the following method, when added to `OrderRepository`, might prove useful:

```
List<TacoOrder> readOrdersByDeliveryZipAndPlacedAtBetween(
    String deliveryZip, Date startDate, Date endDate);
```

Figure 3.2 illustrates how Spring Data parses and understands the `readOrdersByDeliveryZipAndPlacedAtBetween()` method when generating the repository implementation. As you can see, the verb in `readOrdersByDeliveryZipAndPlacedAtBetween()` is `read`. Spring Data also understands `find`, `read`, and `get` as synonymous for fetching one or more entities. Alternatively, you can also use `count` as the verb if you want the method to return only an `int` with the count of matching entities.

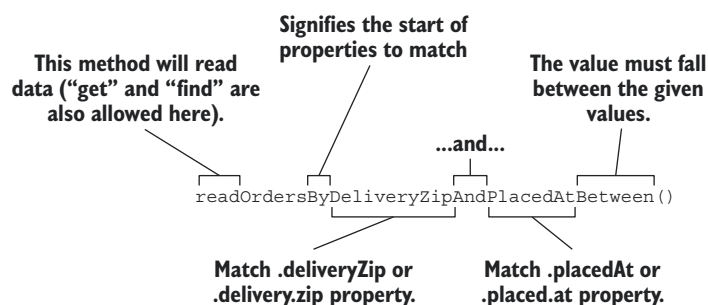


Figure 3.2 Spring Data parses repository method signatures to determine the query that should be performed.

Although the subject of the method is optional, here it says `Orders`. Spring Data ignores most words in a subject, so you could name the method `readPuppiesBy...` and it would still find `TacoOrder` entities, because that is the type that `CrudRepository` is parameterized with.

The predicate follows the word *By* in the method name and is the most interesting part of the method signature. In this case, the predicate refers to two `TacoOrder` properties: `deliveryZip` and `placedAt`. The `deliveryZip` property must be equal to the value passed into the first parameter of the method. The keyword `Between` indicates that the value of `deliveryZip` must fall between the values passed into the last two parameters of the method.

In addition to an implicit `Equals` operation and the `Between` operation, Spring Data method signatures can also include any of the following operators:

- IsAfter, After, IsGreaterThan, GreaterThan
- IsGreaterThanOrEqualTo, GreaterThanOrEqualTo
- IsBefore, Before, IsLessThan, LessThan
- IsLessThanOrEqualTo, LessThanOrEqualTo
- IsBetween, Between
- IsNull, Null
- IsNotNull, NotNull
- IsIn, In
- IsNotIn, NotIn
- IsStartingWith, StartingWith, StartsWith
- IsEndingWith, EndingWith, EndsWith
- IsContaining, Containing, Contains
- IsLike, Like
- IsNotLike, NotLike
- IsTrue, True
- IsFalse, False
- Is, Equals
- IsNot, Not
- IgnoringCase, IgnoresCase

As alternatives for `IgnoringCase` and `IgnoresCase`, you can place either `AllIgnoringCase` or `AllIgnoresCase` on the method to ignore case for all String comparisons. For example, consider the following method:

```
List<TacoOrder> findByDeliveryToAndDeliveryCityAllIgnoresCase(
    String deliveryTo, String deliveryCity);
```

Finally, you can also place `OrderBy` at the end of the method name to sort the results by a specified column. For example, to order by the `deliveryTo` property, use the following code:

```
List<TacoOrder> findByDeliveryCityOrderByDeliveryTo(String city);
```

Although the naming convention can be useful for relatively simple queries, it doesn't take much imagination to see that method names could get out of hand for more complex queries. In that case, feel free to name the method anything you want and annotate it with `@Query` to explicitly specify the query to be performed when the method is called, as this example shows:

```
@Query("Order o where o.deliveryCity='Seattle'")
List<TacoOrder> readOrdersDeliveredInSeattle();
```

In this simple usage of `@Query`, you ask for all orders delivered in Seattle. But you can use `@Query` to perform virtually any JPA query you can dream up, even when it's difficult or impossible to achieve the query by following the naming convention.

Custom query methods also work with Spring Data JDBC but with the following key differences:

- All custom query methods require `@Query`. This is because, unlike JPA, there's no mapping metadata to help Spring Data JDBC automatically infer the query from the method name.
- All queries specified in `@Query` must be SQL queries, not JPA queries.

In the next chapter, we'll expand our use of Spring Data to work with nonrelational databases. When we do, you'll see that custom query methods work very similarly, although the query language used in `@Query` will be specific to the underlying database.

Summary

- Spring's `JdbcTemplate` greatly simplifies working with JDBC.
- `PreparedStatementCreator` and `KeyHolder` can be used together when you need to know the value of a database-generated ID.
- Spring Data JDBC and Spring Data JPA make working with relational data as easy as writing a repository interface.